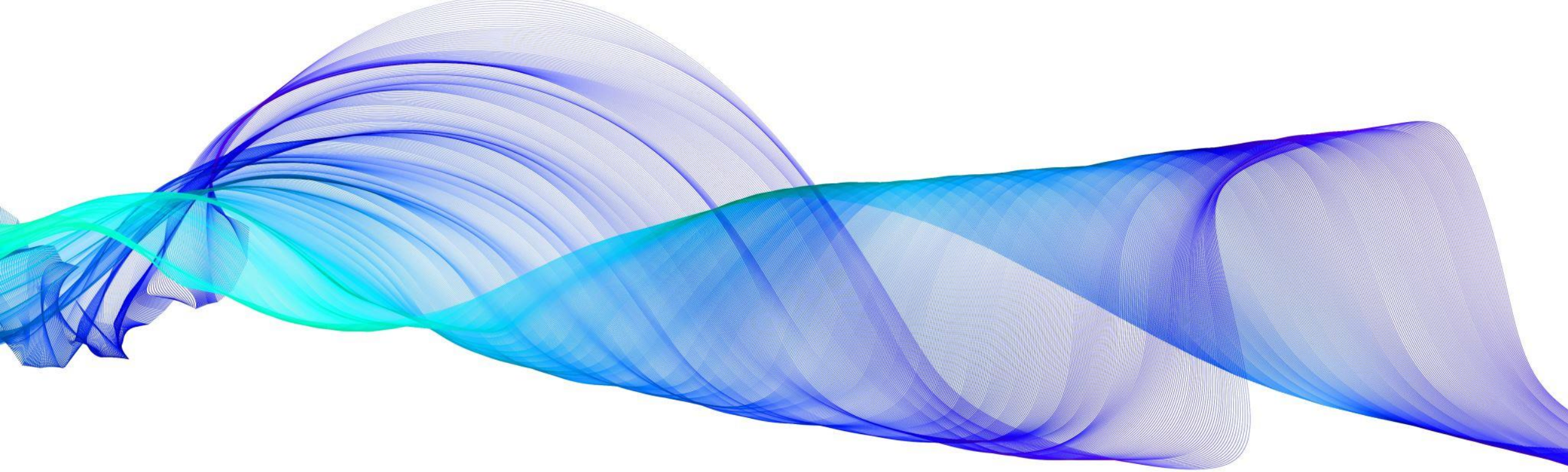




MySQL 8.0

D DATA SCIENCE ACADEMY

▣ 데이터베이스(Database)와 데이터베이스 관리 시스템(DBMS)

- **데이터베이스 (Database)**

- 여러 응용시스템을 통해 액세스가 가능한 공유된 형태(Networked data)의 통합된 데이터의 집합, 즉, 기업이 지속적으로 유지 관리해야 할 데이터의 집합.

- **데이터베이스 관리 시스템(Database Management System)**

- 방대한 양의 데이터를 편리하게 관리하고, 효율적으로 저장하고 검색할 수 있는 환경을 제공하는 소프트웨어

- **데이터베이스 특징**

- DATA의 독립성 (Independency) 유지
- DATA의 무결성 (Integrity) 유지
- DATA의 보안성 (Security)
- DATA의 불일치 (Inconsistency) 배제
- DATA의 중복성 (Redundancy) 감소
- DATA의 공유 (Share)

▣ 관계형 데이터베이스의 이해

• 관계형 데이터베이스의 정의

- 관계는 업무적인 연관성이다. (원래는 하나였는데 두 개로 나뉘었을 때 그 연결고리)
- 관계는 두 테이블 사이에 존재하며, 관계를 맺고 있는 두 테이블 중 반드시 하나는 부모(주체) 테이블이고 반드시 다른 하나는 자식 테이블이다.
- 부모 테이블의 기본키(Primary key)는 자식 테이블의 외래키(Foreign Key)로 전이된다.
- 데이터베이스는 최소한의 의미를 가지는 테이블들로 구성되며 그 테이블들에 있는 필드들로 연결한 것이다. (필드 또한 가장 작은 논리적인 단위로 구분하는 것이 좋다.)

Database Engines Ranking

- DB-Engines Ranking

421 systems in ranking, June 2024

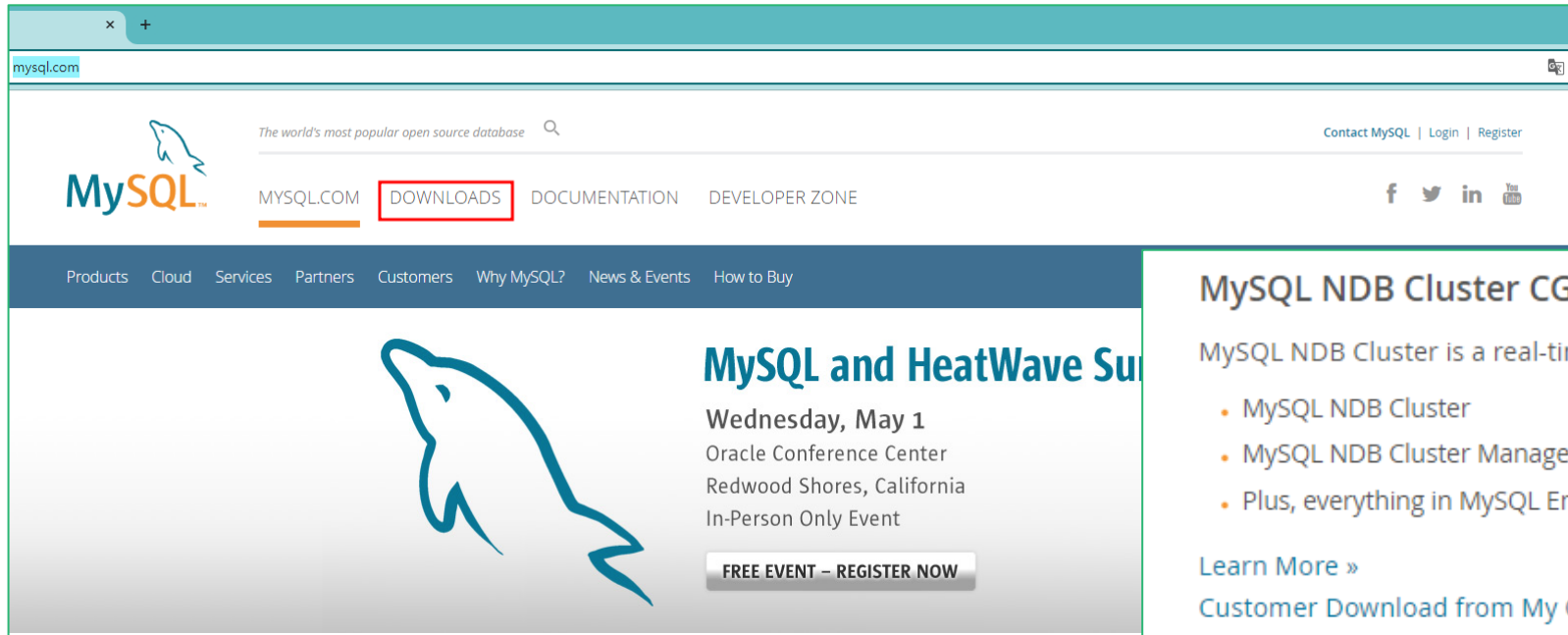
Rank Jun 2024	Rank May 2024	Rank Jun 2023	DBMS	Database Model	Score		
					Jun 2024	May 2024	Jun 2023
1.	1.	1.	Oracle +	Relational, Multi-model ⓘ	1244.08	+7.79	+12.61
2.	2.	2.	MySQL +	Relational, Multi-model ⓘ	1061.34	-22.39	-102.59
3.	3.	3.	Microsoft SQL Server +	Relational, Multi-model ⓘ	821.56	-2.73	-108.50
4.	4.	4.	PostgreSQL +	Relational, Multi-model ⓘ	636.25	-9.30	+23.43
5.	5.	5.	MongoDB +	Document, Multi-model ⓘ	421.08	-0.58	-4.29
6.	6.	6.	Redis +	Key-value, Multi-model ⓘ	155.94	-1.86	-11.41
7.	7.	↑ 8.	Elasticsearch	Search engine, Multi-model ⓘ	132.83	-2.52	-10.92
8.	↑ 9.	↑ 11.	Snowflake +	Relational	130.36	+9.03	+16.23
9.	↓ 8.	↓ 7.	IBM Db2	Relational, Multi-model ⓘ	125.90	-2.56	-18.99
10.	10.	10.	SQLite +	Relational	111.41	-2.91	-19.81

출처 : <https://db-engines.com/en/ranking>

MySQL 설치

다운로드

- Download URL: <https://www.mysql.com/>
- Document URL: <https://dev.mysql.com/doc/refman/8.0/en/>
- 화면 상단의 “DOWNLOADS” 클릭 → 화면 하단의 “MySQL Community(GPL) Downloads” 클릭



MySQL NDB Cluster CGE

MySQL NDB Cluster is a real-time open source transactional database designed for fast, always-on

- MySQL NDB Cluster
- MySQL NDB Cluster Manager
- Plus, everything in MySQL Enterprise Edition

[Learn More »](#)

[Customer Download from My Oracle Support \(MOS\) »](#)

[Trial Download from Oracle edelivery »](#)

[MySQL Community \(GPL\) Downloads »](#)

MySQL 설치

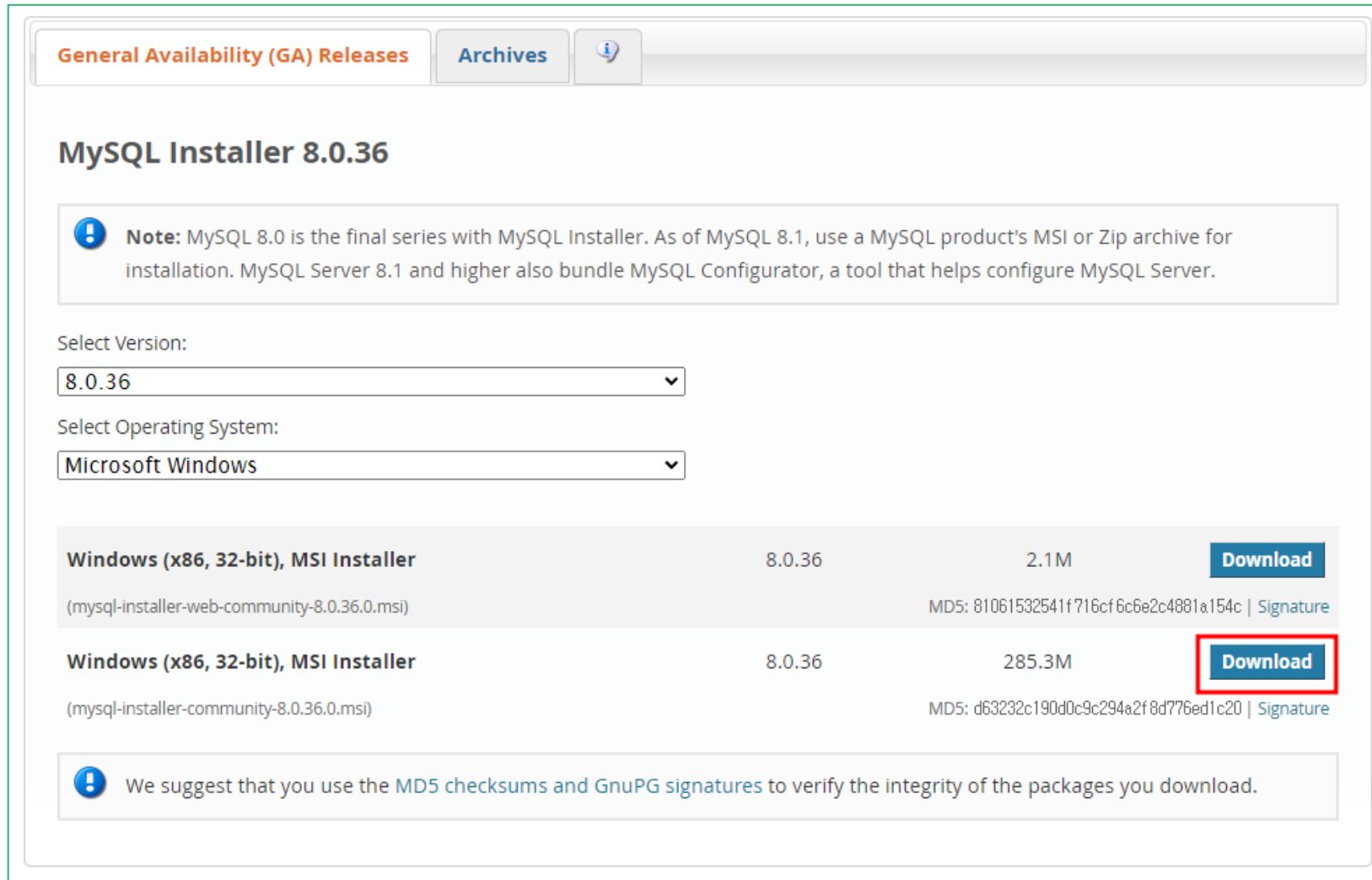
- “MySQL Installer for Windows” 클릭

MySQL Community Downloads

- MySQL Yum Repository
- MySQL APT Repository
- MySQL SUSE Repository
- MySQL Community Server
- MySQL NDB Cluster
- MySQL Router
- MySQL Shell
- MySQL Operator
- MySQL NDB Operator
- MySQL Workbench
- MySQL Installer for Windows
- C API (libmysqlclient)
- Connector/C++
- Connector/J
- Connector/NET
- Connector/Node.js
- Connector/ODBC
- Connector/Python
- MySQL Native Driver for PHP
- MySQL Benchmark Tool
- Time zone description tables
- Download Archives

MySQL 설치

- “Windows(x86, 32-bit), MSI Installer”의 Download 버튼 클릭



General Availability (GA) Releases Archives

MySQL Installer 8.0.36

Note: MySQL 8.0 is the final series with MySQL Installer. As of MySQL 8.1, use a MySQL product's MSI or Zip archive for installation. MySQL Server 8.1 and higher also bundle MySQL Configurator, a tool that helps configure MySQL Server.

Select Version:
8.0.36

Select Operating System:
Microsoft Windows

Windows (x86, 32-bit), MSI Installer (mysql-installer-web-community-8.0.36.0.msi)	8.0.36	2.1M	Download
Windows (x86, 32-bit), MSI Installer (mysql-installer-community-8.0.36.0.msi)	8.0.36	285.3M	Download

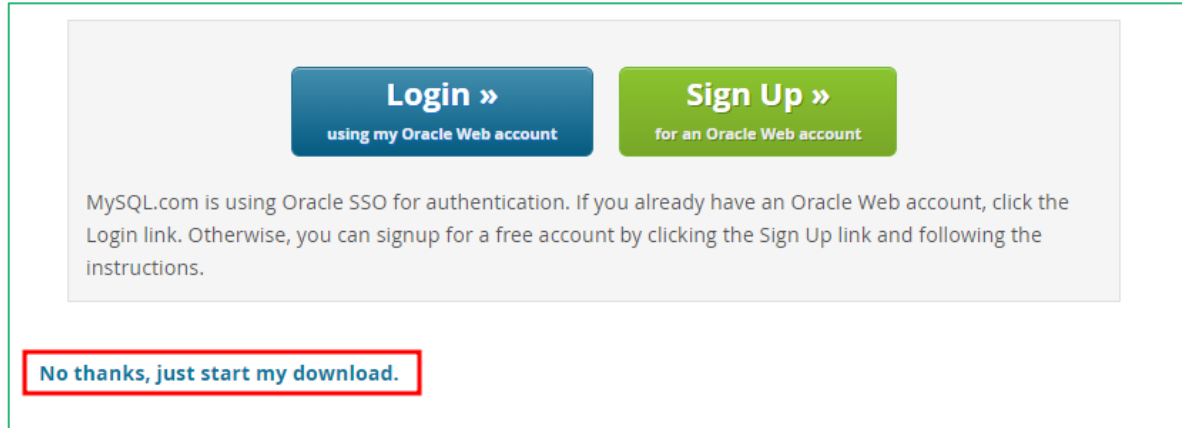
MD5: 81061532541f716cf6c6e2c4881a154c | [Signature](#)

MD5: d63232c190d0c9c294a2f8d776ed1c20 | [Signature](#)

Note: We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.

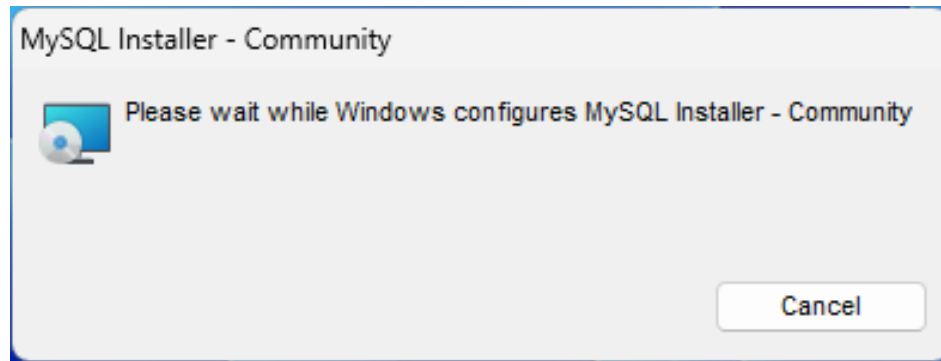
MySQL 설치

- 로그인 여부를 묻는 화면에서 하단의 “No thanks, just start my download.” 클릭
(로그인하지 않고 다운로드를 함)



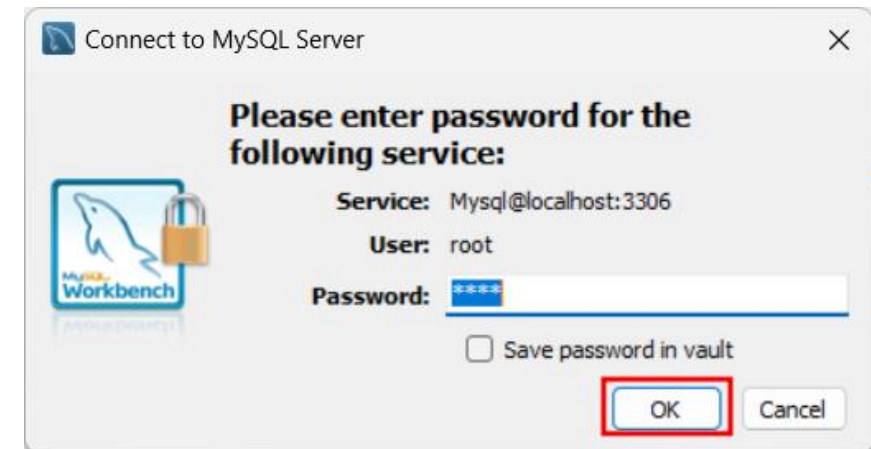
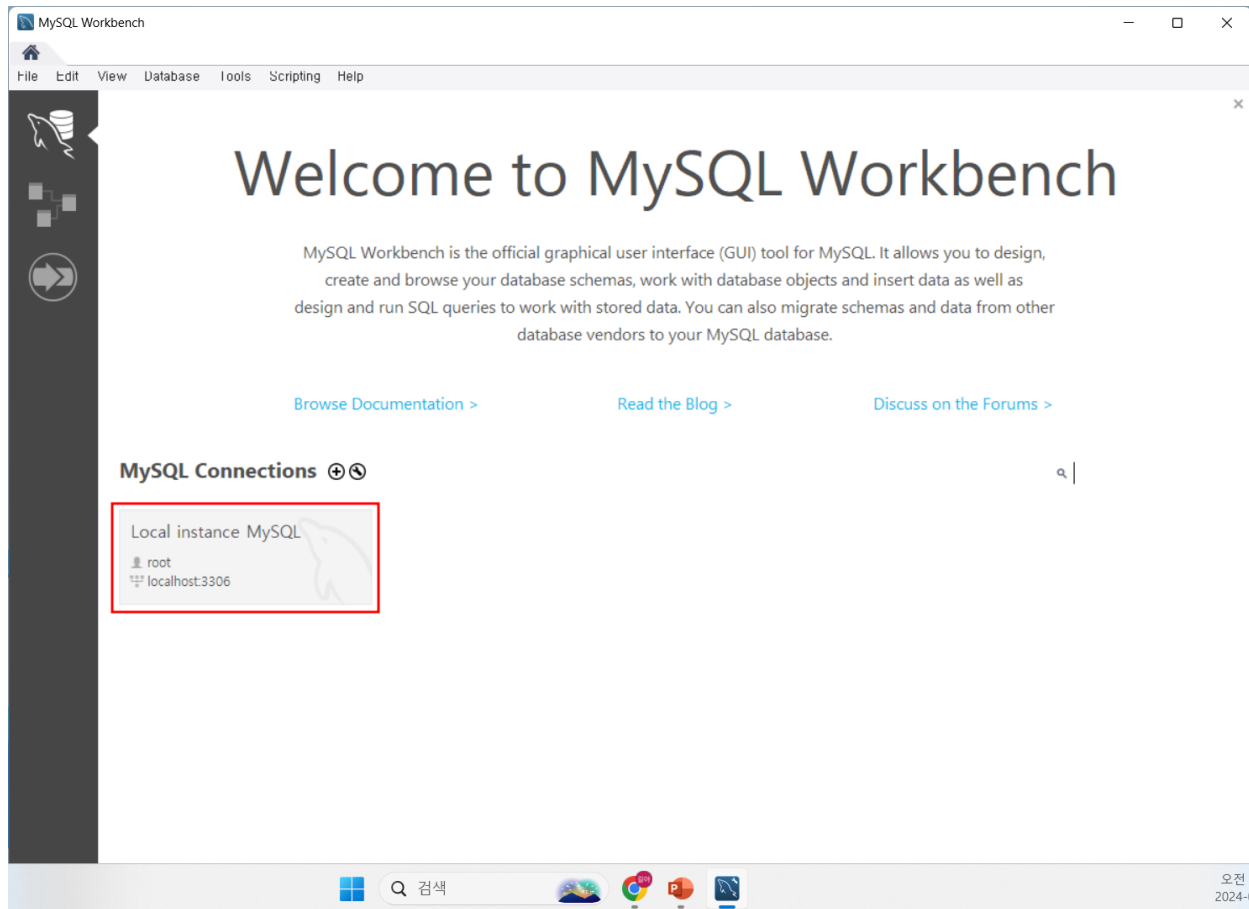
□ MySQL 설치

- 설치
 - 앞서 다운로드 받은 파일을 실행시켜 설치를 진행한다.



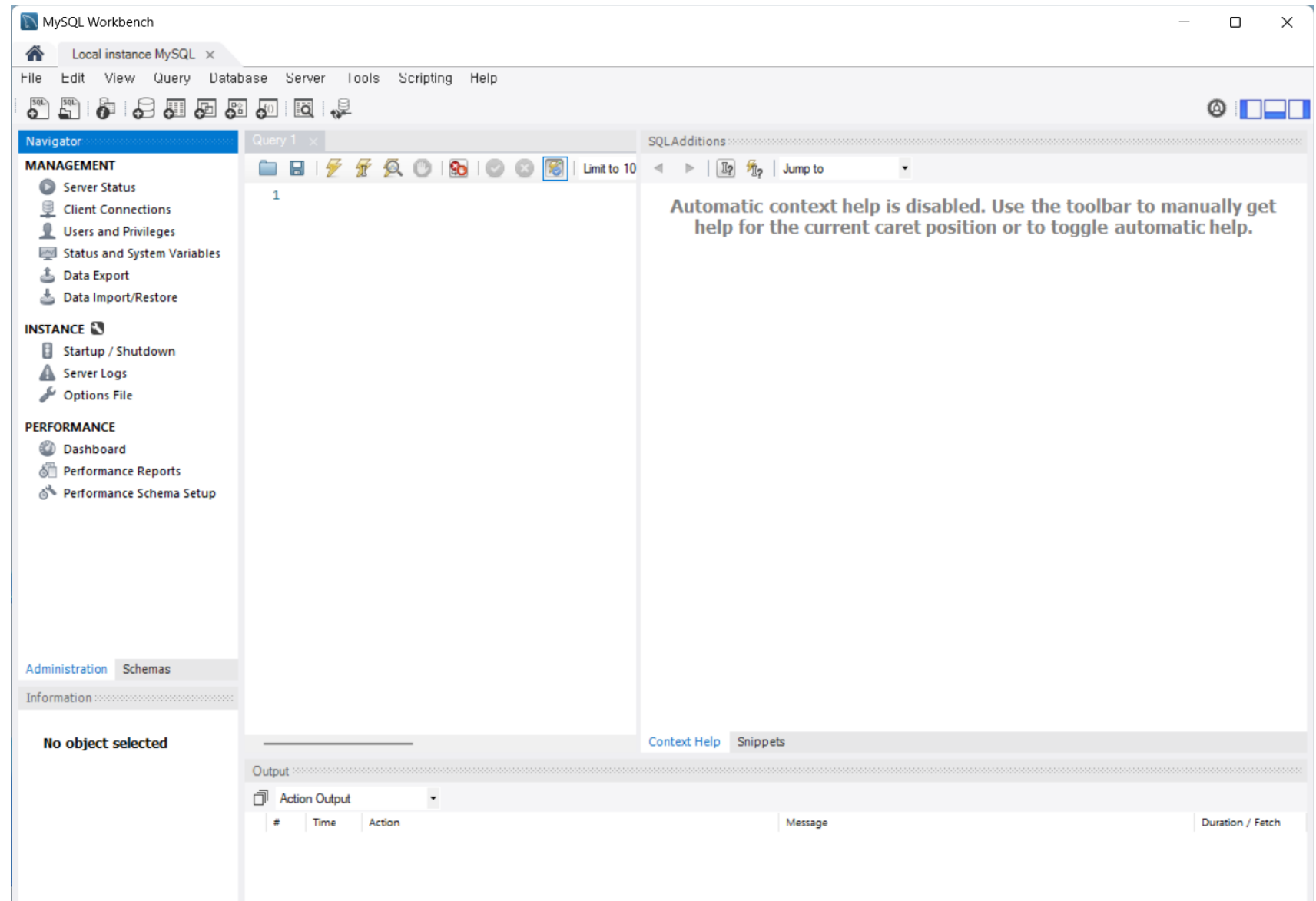
Workbench 설치

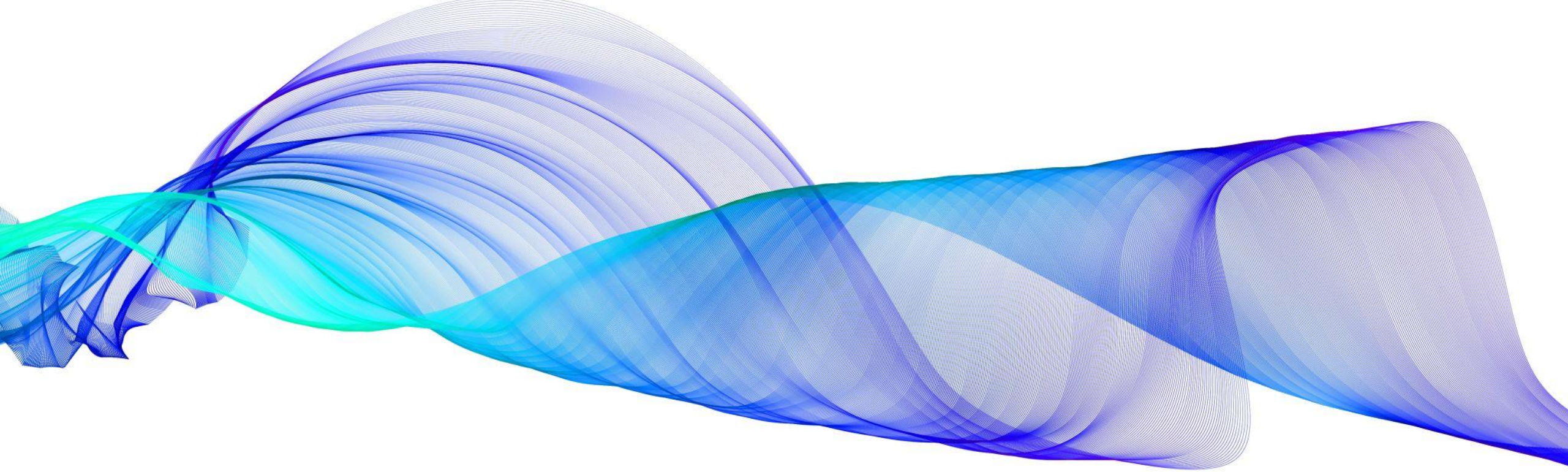
- MySQL Workbench 프로그램 실행 후 접속확인



□ Workbench 설치

- Workbench 실행 화면





SQL 명령어

주석 (Remark)

- 주석

- 프로그램에 설명을 달거나 특정 보드 부분의 실행을 막고자 할 때 사용된다.
- MySQL은 2가지 주석이 제공된다.

① 한 줄 주석 : -- 주석의 내용

- -- 를 이용한 주석은 기호 뒤에 한 칸 띄어쓰기를 해야 주석으로 인식한다.

② 여러 줄 주석: /* 주석의 내용 */

▣ SQL(Structured Query Language)

- SQL
 - 관계형 데이터베이스 관리 시스템에서 데이터를 관리하기 위해 사용되는 표준 프로그래밍 언어(Language)
 - 데이터베이스 스키마 생성 및 수정, 테이블 관리, 데이터 추가, 수정, 삭제, 조회 등, 데이터베이스와 관련된 거의 모든 작업을 위해 사용되는 언어
 - 데이터베이스마다 문법에 약간의 차이가 있지만, 표준 SQL을 기본으로 한다.
- SQL의 종류

종류	설명	종류
DDL(Data Definition Language)	데이터 정의어	데이터베이스의 객체(데이터베이스, 테이블, 사용자, 뷰, 인덱스)등을 생성, 삭제, 변경해주는 명령어 CREATE, ALTER, DROP, TRUNCATE
DML(Data Manipulation Language)	데이터 조작어	테이블의 각 레코드의 값을 검색, 등록, 수정, 삭제하는 명령어 SELECT, INSERT, UPDATE, DELETE
DCL(Data Control Language)	데이터 제어어	데이터베이스의 사용권한을 관리 GRANT, REVOKE, COMMIT, ROLLBACK ※ TCL(Transaction Control Language) DCL중 COMMIT, ROLLBACK은 트랜잭션을 완료하거나 복구하는 명령어

▣ 예약어(Reserved Word)

• 예약어

- 키워드라고도 하며, DB 서버가 특정 명령의 사용을 위해 미리 그 사용 용도를 정해 놓은 단어를 말한다.
- 사용자가 테이블이나 테이블 내의 컬럼 등을 생성할 때 예약어를 사용하지 않는 것이 좋다.
- 예약어와 다르게 식별자는 객체 생성 시 사용가능한 단어를 말한다.
- MySQL 버전마다 예약어가 추가되거나 삭제되었기 때문에 자세한 내용은 아래 URL에서 확인할 수 있다.

• 예약어의 확인

- URL: <https://dev.mysql.com/doc/mysqld-version-reference/en/keywords-8-0.html>

▣ 식별자(Identifier)

- 식별자
 - DBMS 사용자가 필요에 의해서 생성한 개체들에 붙이는 이름을 식별자(Identifier)라고 한다.
 - 사용자가 작성하는 객체의 종류에는 데이터베이스, 테이블, 인덱스, 열 이름, 뷰, 스토어드 프로시저, 트리거 등이 있다.
- 식별자 명명 규칙 (p195)
 - 알파벳 대소문자, 숫자, 특수문자 중 \$, _(Under score) 사용가능. 영문자 대문자를 사용해도 소문자로 생성된다.
 - 숫자로 시작하는 이름을 가진 식별자는 사용 가능하지만 숫자로만 식별자를 만들 수는 없다. (가능하면 영문자 사용 권장)
 - 개체의 이름은 최대 64자 이내여야 한다.
 - 예약어를 식별자로 사용할 수 없다.
 - 식별자 중간에 공백을 넣으면 안되며, 공백이 반드시 필요한 경우 백틱(' ')으로 묶어서 사용한다.

Database의 생성

MySQL Database 관리 명령어

- MySQL에서는 데이터베이스(=스키마)를 생성한 후 해당 데이터베이스 내에 테이블을 생성하고, 생성된 테이블에 레코드를 저장하여 관리
- MySQL의 데이터베이스 관련 명령어는 아래와 같다.

-- 1. 현재 생성된 데이터베이스의 종류를 나열

SHOW DATABASES;

	ABC Database ▼
1	information_schema
2	mydb
3	mysql
4	performance_schema
5	sakila
6	sys
7	world

-- 2. 한글세트를 저장할 수 있는 데이터베이스 생성

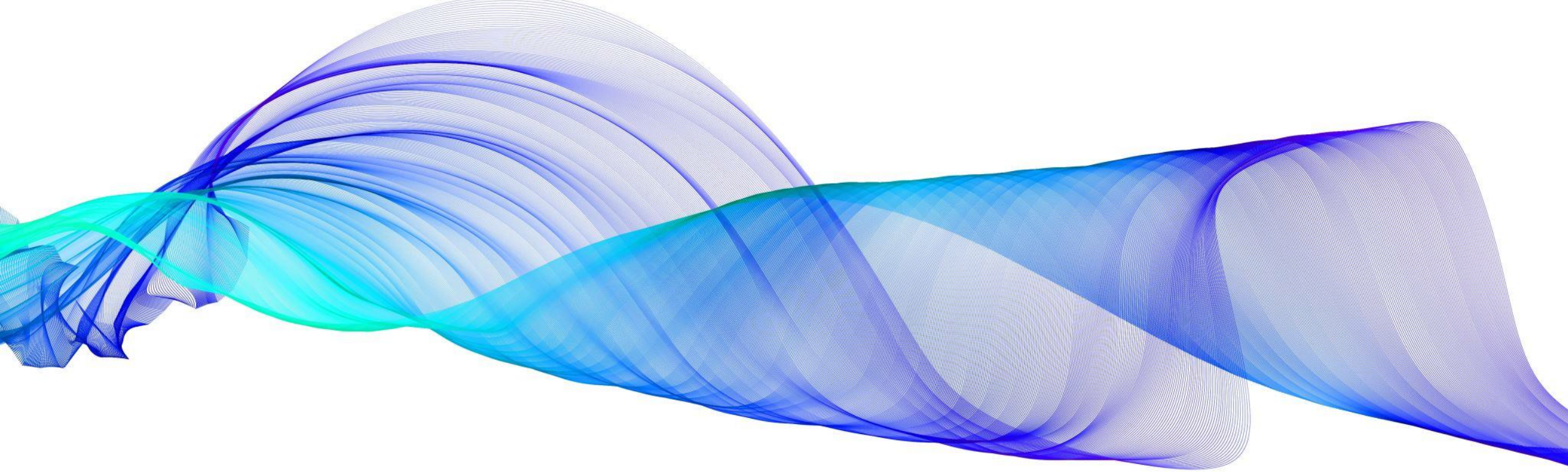
CREATE DATABASE 'hr' DEFAULT CHARACTER SET UTF8MB3;

-- 3. 데이터베이스 삭제

DROP DATABASE 'hr';

-- 4. 데이터베이스 사용

USE 'hr';



Database Object

□ Database Object

- **DDL(Data Definition Language)**

- DDL은 MySQL 데이터베이스에서 관리하는 개체들을 정의하고, 관리할 때 사용되는 명령어.
- CREATE, ALTER, DROP, TRUNCATE 가 있다.

- **MySQL Object의 종류**

종류	종류
스키마(Schema)	MySQL에서 스키마는 데이터베이스와 같은 용어이다. 데이터베이스 내의 객체의 구조와 제약 조건 등을 정의 한 것 이라고 볼 수 있으며, 사용자의 따라 접근을 제한하는 등의 관리가 가능한 집합을 말한다. 하나의 스키마(데이터베이스) 내에 여러 개의 테이블을 생성 할 수 있고, 각각의 스키마 별로 문자셋을 구분하여 지정할 수 있다. > CREATE SCHEMA sample default character set UTF8MB4; -- 아래와 동일한 문장 > CREATE DATABASE sample default character set UTF8MB4;

□ Database Object

종류	종류
테이블(Table)	데이터베이스의 기본 저장 단위로, 행과 열로 구성된 데이터 구조이다. 각 열에는 관리해야 하는 값이 들어가 있으며 데이터의 무결성 유지를 위한 5가지의 제약조건이 사용된다.
인덱스(Index)	인덱스란 데이터베이스 내의 테이블에서 원하는 데이터를 좀 더 빨리 찾아줄 수 있게끔 데이터의 위치 정보를 모아 놓은 데이터베이스 객체
뷰(View)	가상테이블. 테이블과 유사하며 테이블을 통해서 작성되지만 실제 데이터를 가지고 있지 않으며, 실제 데이터를 가진 테이블을 View를 통해 조회할 수 있다.

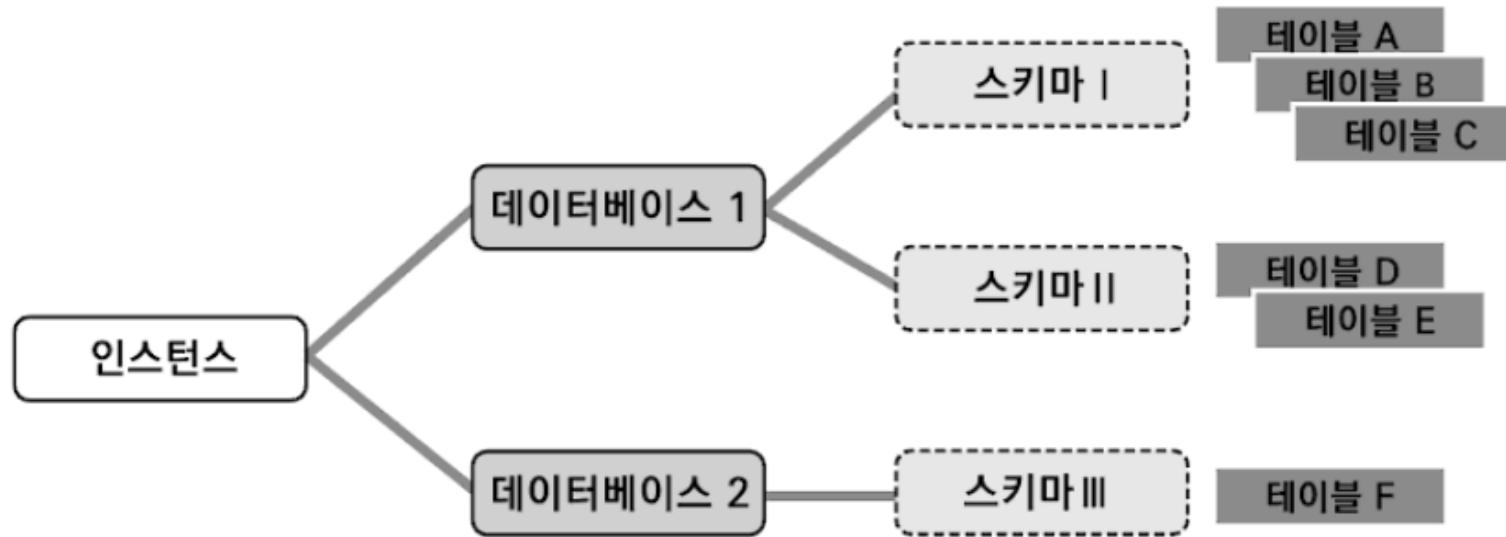
□ Database Schema

- Database Schema란?

- 데이터베이스의 테이블 구조 및 형식, 관계 등의 정보를 형식 언어(formal language)로 기술한 것
- 관계형 데이터베이스를 사용하여 데이터를 저장할 때 가장 먼저 할 일은 데이터의 공통 속성을 식별하여 컬럼(Column)으로 정의하고, 테이블(Table)을 만드는 것
- 통상적으로 하나의 테이블이 아닌 여러 개의 테이블로 만들고, 각 테이블 구조, 형식, 관계를 정의함
- 이를 스키마라고 하며, 일종의 데이터베이스 설계도로 이해하면 됨
- 데이터베이스마다 스키마를 만드는 언어가 존재하며, 해당 스키마만 있으면 동일한 구조의 데이터베이스를 만들 수 있음
(데이터베이스 백업과는 달리 데이터 구조만 동일하게 만들 수 있음)

Database Schema

- ANSI 표준 데이터베이스 계층구조 (4계층 구조)

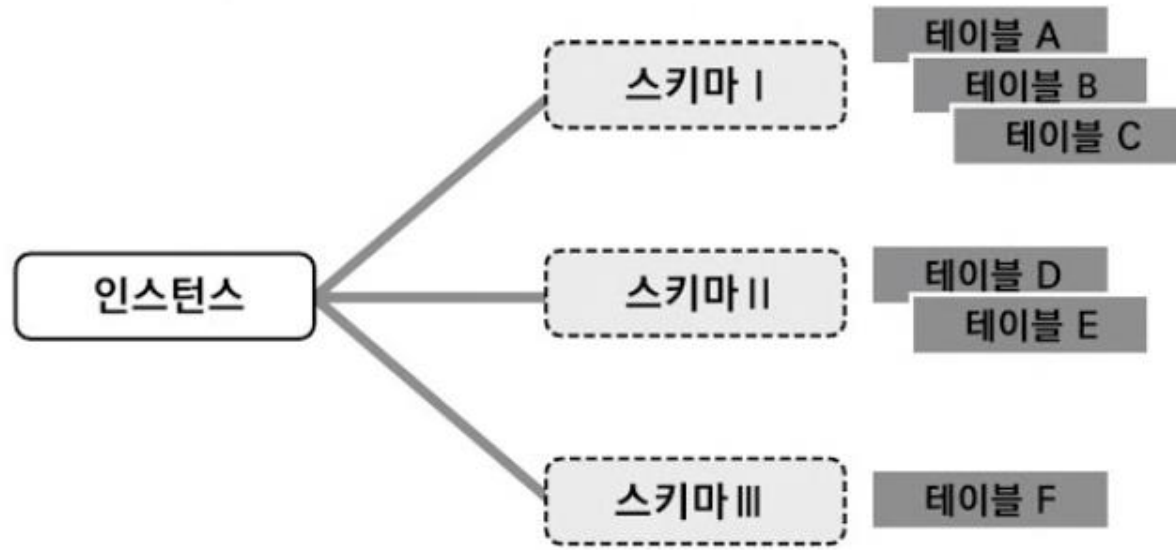


- ① 인스턴스: DBMS가 동작할 때의 단위, OS 입장에서는 '프로세스', DBMS에 따라서는 '서버 프로세스' 또는 '서버'로 부르기도 한다.
- ② 데이터베이스: 오라클의 경우 작동하는데 필요한 모든 파일을 의미
- ③ 스키마: 데이터베이스의 구조와 제약 조건을 정의한 것, 폴더와 같은 개념이라고 이해할 수 있으며 스키마도 폴더처럼 사용자가 자유롭게 만들 수 있고, 사용자에게 따라 접근을 제한하는 등의 권한 관리가 가능.
여러 객체(테이블, 뷰 등)를 담을 수 있다.
- ④ 테이블: 실제 데이터가 담기는 영역

출처: https://hue9010.github.io/db/mysql_schema/

Database Schema

- MySQL의 데이터베이스 계층구조



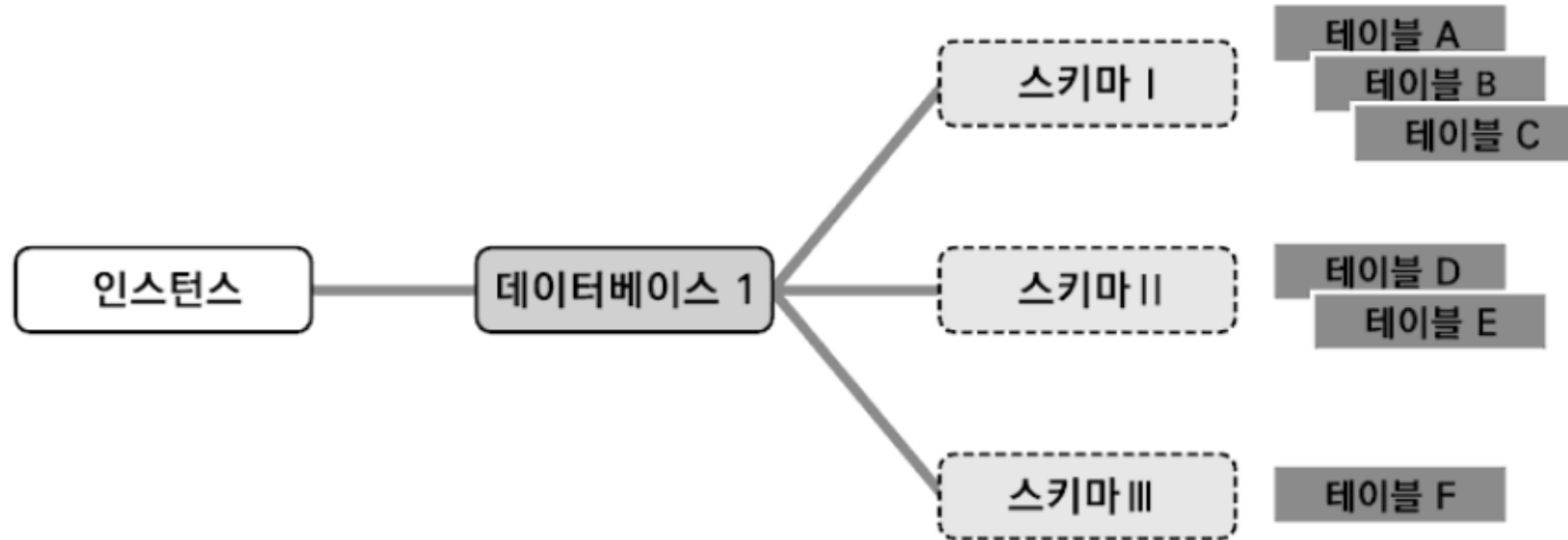
- MySQL은 3계층 구조를 사용하며 데이터베이스가 없다. 데이터베이스와 스키마를 동일하게 취급한다.

```
CREATE SCHEMA sample default character set UTF8MB4; -- 아래와 동일
```

```
CREATE DATABASE sample default character set UTF8MB4;
```

Database Schema

- Oracle 의 데이터베이스 계층구조



- 4계층 구조를 따르지만 하나의 인스턴스에 하나의 데이터베이스만 만들 수 있다.

□ Schema

• Schema란?

- MySQL에서 스키마는 데이터베이스와 같은 용어이다.
- 데이터베이스 내의 객체의 구조와 제약 조건 등을 정의 한 것 이라고 볼 수 있으며, 사용자의 따라 접근을 제한하는 등의 관리가 가능한 집합을 말한다.
- 하나의 스키마(데이터베이스) 내에 여러 개의 테이블을 생성 할 수 있고, 각각의 스키마 별로 문자셋을 구분하여 지정할 수 있다.

• Schema 생성

```
CREATE SCHEMA 스키마명 default character set UTF8MB4;      -- 아래와 동일한 문장
CREATE DATABASE 스키마명 default character set UTF8MB4;
```

• Schema 삭제

```
DROP DATABASE 스키마명;
```

• Schema 조회

```
SHOW DATABASE 스키마명;
```

• Schema 사용

```
USE 스키마명;
```

□ Table

• 테이블의 구성

- 관계형 데이터베이스는 2차원적인 표(Table)를 이용해서 데이터 상호 관계를 정의
- 개체(Entity)와 관계(Relationship)를 모두 릴레이션(Relation)이라는 표(Table)로 표현하기 때문에 개체를 개체 릴레이션과 관계 릴레이션으로 표현한다.

• 개체 릴레이션과 관계 릴레이션

- 개체 릴레이션: 데이터베이스에서 표현하고자 하는 독립적인 개체(Entity)를 나타내는 릴레이션.
각 개체는 고유한 식별자(기본키)를 가지며, 해당 개체의 속성(Attribute)들로 구성된다.
- 관계 릴레이션: 두 개 이상의 개체 간의 관계(Relationship)를 나타내는 릴레이션.
관계 릴레이션에는 두 개체의 기본키가 외래키로 포함되며, 이를 통해 두 개체 간의 연관성을 표현한다.
- 관계형 데이터베이스에서는 이러한 개체 릴레이션과 관계 릴레이션을 이용하여 복잡한 데이터를 체계적으로 관리하고, 다양한 쿼리(Query)를 수행.

Table

- 관계형 데이터베이스

<employees>
Entity

Attribute

Tuple

Relation Schema

Relation Instance

	employee_id	first_name	last_name	email	phone_number
1	100	Steven	King	SKING	515.123.4567
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568
3	102	Lex	De Haan	LDEHAAN	515.123.4569
4	103	Alexander	Hunold	AHUNOLD	590.423.4567
5	104	Bruce	Ernst	BERNST	590.423.4568
6	105	David	Austin	DAUSTIN	590.423.4569
7	106	Valli	Pataballa	VPATABAL	590.423.4560
8	107	Diana	Lorentz	DLORENTZ	590.423.5567
9	108	Nancy	Greenberg	NGREENBE	515.124.4569
10	109	Daniel	Faviet	DFAVIET	515.124.4169

□ Table

• 관계형 데이터베이스 용어

용어	설명
릴레이션(Relation)	관계형 데이터베이스에서 정보를 구분하여 저장하는 기본 단위이다. 쉽게 말해 데이터베이스의 테이블(Table)을 의미
릴레이션 스키마 (Relation Schema)	릴레이션의 구조를 정의한 것으로, 릴레이션의 이름과 포함된 모든 속성(Attribute)의 이름으로 정의하는 논리적 구조
릴레이션 인스턴스 (Relation Instance)	어느 한 시점에 릴레이션에 존재하는 튜플(Tuple)들의 집합을 의미
튜플(Tuple)	릴레이션을 구성하는 각각의 행을 의미하며, 파일 구조에서는 레코드(Record)에 해당
어트리뷰트(Attribute)	데이터베이스를 구성하는 가장 작은 논리적 단위로, 개체의 특성을 기술한다. 파일 구조에서는 데이터 항목 또는 데이터 필드(Data Field)에 해당

□ Index

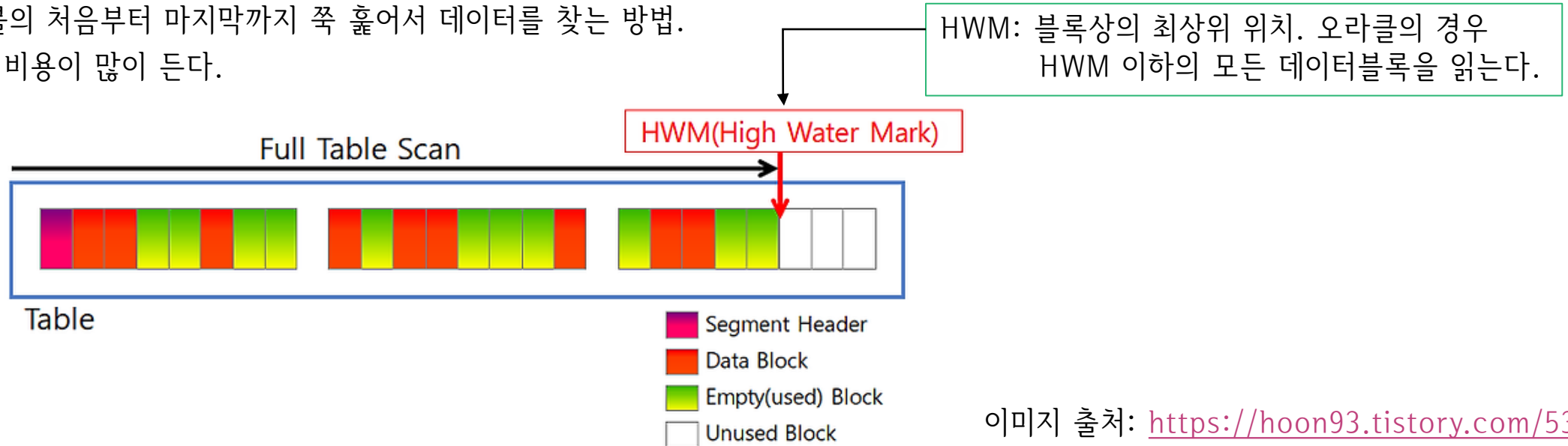
• Index의 개념

- 인덱스란 데이터베이스 내의 테이블에서 원하는 데이터를 좀 더 빨리 찾아줄 수 있게끔 데이터의 위치 정보를 모아 놓은 데이터베이스 내의 객체(Object)이다.
- 인덱스는 항상 정렬된 상태를 유지한다.
- 테이블의 Primary Key나 Unique를 설정하면 해당 컬럼을 이용하여 자동으로 인덱스가 생성된다.

• 검색 방법

1) Full Scan(=Table scan)

- 테이블의 처음부터 마지막까지 쭉 훑어서 데이터를 찾는 방법.
- 시간, 비용이 많이 든다.



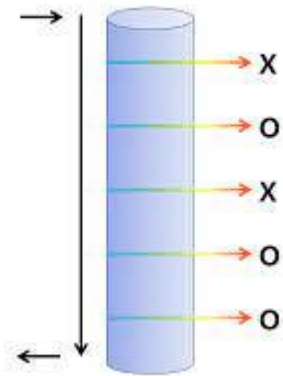
이미지 출처: <https://hoon93.tistory.com/53>

□ Index

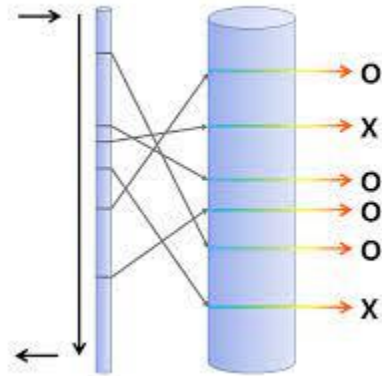
• 검색 방법

2) Index Scan

- 목차의 개념, 즉 목차를 이용하여 원하는 데이터를 빠르게 찾을 때 사용하게 된다.
- 인덱스를 구성하는 컬럼의 값을 기반으로 데이터를 추출
- 인덱스에 존재하지 않는 컬럼의 값이 필요한 경우에는 현재 읽은 레코드 식별자를 이용하여 테이블을 액세스 해야 한다.
그러나 SQL문에서 필요로 하는 모든 컬럼이 인덱스 구성 컬럼에 포함된 경우 테이블에 대한 액세스는 발생하지 않는다.



Full Table Scan



Index Scan

이미지 출처: <https://hoon93.tistory.com/53>

□ Index

• Index 생성 기준

- SQL문에서 WHERE 조건절에 자주 사용되는 컬럼이 인덱스 생성 대상이 된다.
- 개수는 사용형태에 따라 적절하게 생성한다.
- 너무 많은 인덱스는 오히려 성능이 감소된다.
- 검색은 인덱스가 많을수록 좋고, UPDATE, DELETE, INSERT는 인덱스가 적을수록 좋다.
- 빈번하게 변경되지 않는 테이블에 적용된다. (컬럼 분포도가 2~4%인 경우 (10% 기준))
- 찾으려는 데이터가 총 500페이지에서 400페이지라면 컬럼 분포도가 매우 높은 것이므로 인덱스를 사용하는 것이 좋지 않다.
- 작은 테이블보단 큰 테이블에 인덱스를 생성한다.

• Index 장점

- 검색 속도가 빨라진다.
- 특정 쿼리를 반복적으로 사용한다면 해당 쿼리의 부하가 줄어들어 시스템 성능이 향상된다.

• Index 단점

- 추가적으로 index 개체가 저장될 추가 공간이 필요하다. 데이터 베이스의 약 10% 가량
- index를 처음 생성할 때 시간이 소요된다.
- 데이터의 UPDATE, DELETE, INSERT가 빈번하게 일어나는 테이블의 경우에는 오히려 index의 update의 시간이 더 필요하게 되므로 성능이 더 나빠질 수도 있다.

□ Index

• MySQL의 Index를 나누는 방법-1

1) Clustered index (= Clustered Index)

- 사전과 같이 데이터 자체가 정렬 된 상태로 생성되는 index로 테이블당 한 개만 생성된다.
- 테이블을 생성할 때 Primary Key를 지정하면 자동으로 해당 컬럼열에 클러스터드 index가 생성된다.
- Primary Key가 아닌 컬럼이 Not Null, Unique 속성을 모두 가지게 되면 클러스터드 index가 생성된다.
- “show index from 테이블명;” 명령을 실행했을 때 Key_name의 값이 “PRIMARY”이면 클러스터드 인덱스이다.

2) Non Clustered index (=Secondary Index)

- 큰 책의 뒷 부분에 있는 “찾아보기”와 같은 개념의 index.
- 테이블을 생성할 때 Unique 제약조건을 지정하면 자동으로 해당 컬럼열에 Non Clustered index가 생성된다.

• MySQL의 Index를 나누는 방법-2

1) Unique Index

- 인덱스 값들이 서로 중복되지 않는 인덱스, 컬럼의 제약조건을 PK나 Unique를 지정하면 Unique Index가 생성된다.
- “show index from 테이블명;” 명령을 실행했을 때 Non_unique 열이 0이면 Unique Index이다.

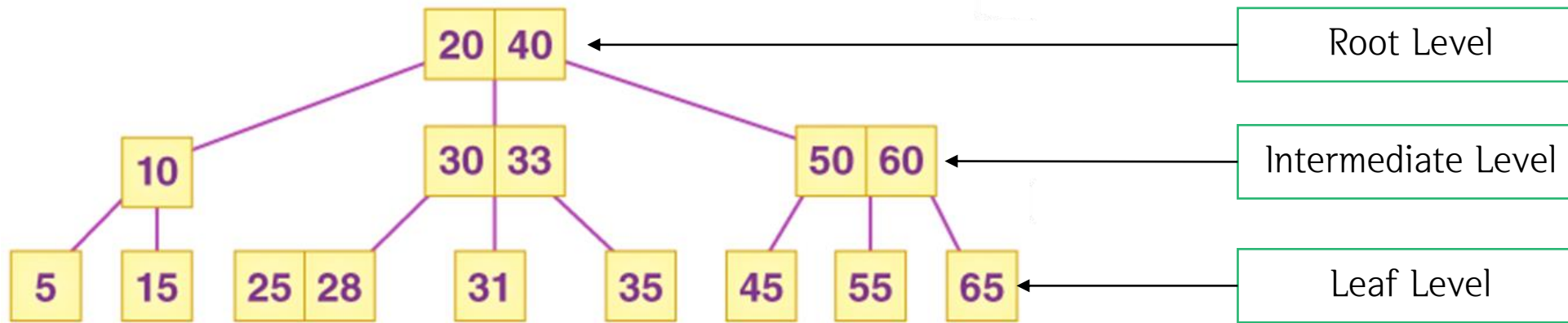
2) Non_unique Index

- PK나 UQ 제약조건이 없는 컬럼을 이용해 인덱스가 생성되었을 때 Non_unique Index이다.

Index

- **B-Tree(Balanced Tree)**

- index 개체는 Root 노트의 하위 Node의 구조가 균형 잡힌 형태의 자료구조인 Balanced Tree 구조이다.



이미지 출처: <https://byjus.com/gate/b-tree-notes/>

▣ 뷰(View)

• 뷰의 개념

- 뷰(view)는 데이터베이스에 존재하는 일종의 가상 테이블로 사용하는 사용자 입장에서는 테이블과 동일하게 사용하는 개체이다.
- 실제 테이블처럼 행과 열을 가지고 있지만, 실제로 데이터를 저장하고 있지는 않으며 실제 존재하는 테이블로부터 조회된 테이블을 창을 통해 보듯이 동작한다.
- 뷰로 생성된 테이블의 컬럼값은 수정을 할 수 있으나, 권장하지 않으며 주로 SELECT로 조회한다.

• 뷰의 장점

- 중요한 개인정보는 뷰에서 제외시킬 수 있으므로 보안에 도움이 된다.
- 조인이나 서브쿼리 등으로 일반적인 쿼리는 복잡하지만 복잡한 쿼리문을 뷰로 작성하면 더 단순하게 사용할 수 있다.

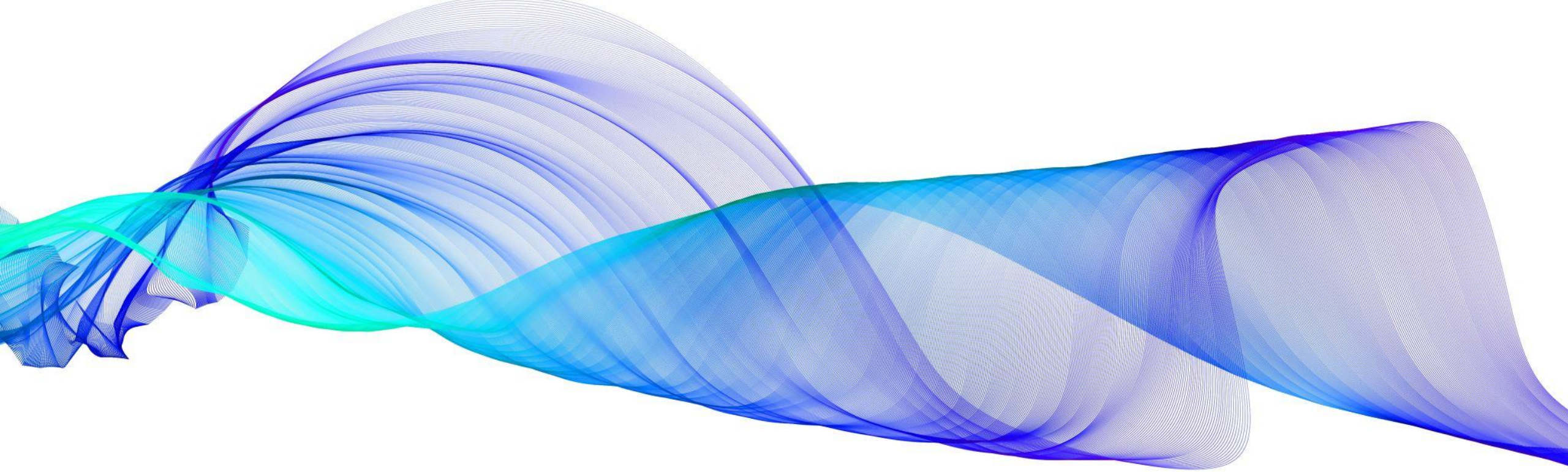
• 뷰의 생성 및 사용

<View의 생성>

```
CREATE VIEW view_name  
AS  
    SELECT 구문;
```

<View의 사용>

```
SELECT 컬럼명 FROM view_name;
```



DML(Data Manipulation Language)

□ DML

- **DML(Data Manipulation Language)**

- 데이터 조작 언어(영어: Data Manipulation Language, DML)은 데이터베이스 사용자 또는 응용 소프트웨어가 컴퓨터 데이터베이스에 데이터를 검색, 등록, 삭제, 갱신을 하기 위한 데이터베이스 언어이다.
- 데이터 조작 언어는 데이터베이스 공급업체(벤더) 간에 언어 및 기능에서 많은 차이를 가진다.
- DML 명령의 아래 4종류가 있으며 이것을 CRUD(Create, Retrieve(Read), Update, Delete)라고 한다.

- **종류**

- INSERT - 삽입 (등록)
- SELECT - 검색 (질의)
- UPDATE - 업데이트 (수정)
- DELETE - 삭제

□ SELECT

- SELECT

- 데이터베이스의 테이블로부터 원하는 정보 조회
- 기초 : 명령문은 대, 소문자를 구별하지 않는다.
- 주석: --, /* */ 로 주석처리 할 수 있다.

- 기본 문법

< 테이블 전체 내용 확인>

SELECT * FROM 테이블명;

<특정 컬럼만 선택해서 확인>

SELECT 컬럼명, 컬럼명... FROM 테이블명;

<중복된 Row 데이터 제거 조회 (default: ALL)>

SELECT [ALL | DISTINCT] 컬럼명 FROM 테이블명

▣ SELECT

- 기본문법

<조건을 이용하여 검색>

SELECT 컬럼명 FROM 테이블명 WHERE 조건문;

<특정 컬럼을 기준으로 정렬하여 조회 (default: ASC)>

SELECT 컬럼명 FROM 테이블명 ORDER BY 컬럼명 [ASC | DESC];

<원하는 개수 조회 (LIMIT 가져올 개수, LIMIT 건너뛸 개수, 가져올 개수)>

SELECT 컬럼명 FROM 테이블명
LIMIT n1, n2;

<Alias 적용하여 컬럼 명의 별칭 사용>

-- 별칭은 문자열로 지정하며 문자열 내에 특수문자가 포함될 경우 Quotation으로 처리

SELECT 컬럼명 AS '별칭' FROM 테이블명;

□ 연산자

종류	연산자	설명	종류	연산자	설명
비교연산자	<, >	작다, 크다	논리연산자	AND, &&	양쪽의 두 부울식이 참일 때
	<=, >=	작거나 같다, 크거나 같다		OR,	부울 표현식이 하나라도 참일 때 참
	=	같다		NOT	부울 연산 결과의 반대 실행
	!=, <>	같지 않다		[NOT] BETWEEN	특정 범위 내에 피연산자가 존재할 때
	!<, !>	작지 않다, 크지 않다		[NOT] IN	리스트 중 하나라도 포함될 경우
사칙연산자	+, -, *, /	더하기, 빼기, 곱하기, 나누기		[NOT] LIKE	피연산자가 패턴과 일치할 경우
	%, mod	나머지		[NOT] EXISTS	하위 쿼리에 행이 포함되면
				SOME	비교 집합 중 일부가 참인 경우 참
				ALL	모든 비교 집합이 참일 경우
				ANY	비교집합 어느 하나라도 참일 때

▣ 변수의 종류

- **MySQL 변수의 종류**

- 사용자 정의 변수
- 지역 변수
- 시스템 변수

- **사용자 정의 변수**

- 사용자가 프로그램 상 필요한 변수를 선언하고 사용하는 변수
- SET을 이용하여 선언하고 사용한다.

- **지역변수(Local Variable)**

- Stored Procedure에서 DECLARE 문으로 선언하고 초기화 된 변수를 지역변수라 한다.
- 지역변수는 변수가 선언된 곳의 BEGIN ~ END 블록내에서만 사용할 수 있다.
- 초기값을 설정할 때에는 DEFAULT 값; 과 같이 초기화 시킬 수 있으며 DEFAULT가 없으면 초기값은 NULL이다.

▣ 변수 (시스템 변수)

• 시스템 변수

- MySQL 서버는 메모리나 작동방식을 초기화하고, 접속된 사용자를 제어하기 위한 설정파일(my.ini or my.cnf)이 있는데 그 내용을 읽어 저장하고 관리하는데 이러한 값을 담은 변수를 시스템 변수라고 한다.
- 시스템 변수는 Var Scope 옵션이 존재하며 이는 변수의 적용 범위를 나타내는 것으로 서버 전체에 영향을 미치는 글로벌 변수와 클라이언트의 커넥션에만 적용되는 세션 변수가 있다.

<시스템 변수 확인>

SHOW VARIABLES;

SHOW GLOBAL VARIABLES;

▣ 변수 (시스템 변수)

- 글로벌 변수 (Global Variable)
 - 서버 전체에 영향을 미치는 변수
 - 주로 서버 자체의 설정에 관한 값을 가진다.
ex) InnoDB 버퍼풀의 크기, MYISAM의 키 캐시크기 등
 - my.ini 파일에 등록된 변수의 값들이 대부분 글로벌 변수이다.
- 세션 변수(Session Variable)
 - MySQL 클라이언트가 서버에 접속할 때 부여되는 옵션의 기본값을 제어할 때 사용
 - 각 클라이언트마다 다른 값으로 변경가능
 - 이 값은 값을 변경할 수 있다.

▣ 변수 (시스템 변수)

• 정적 변수와 동적 변수

- 서버가 구동되면 my.ini 파일에서 설정 정보를 읽어 메모리에 적재하게 되는데 이렇게 읽어 들여 메모리에 업로드된 변수를 동적변수, 저장장치에 저장되어 있는 상태의 변수를 정적변수라고 한다.
- MySQL 서버가 실행중인 상태에서는 시스템 변수의 값이 메모리에 로딩된 상태이므로 이 상태에서 값을 수정해도 설정파일(my.ini)에 적용되는 것이 아니므로 다시 접속할 때에는 적용되지 않는다.
- my.ini에 등록되어 있는 시스템 변수들의 값은 변경을 하더라도 서버를 재실행하기 전까지는 적용되지 않는다
- 현재 수정한 값을 설정파일에 적용하려면 set persist 명령을 통해 파일 내용의 변경이 가능하다.

▣ 변수 (사용자 정의 변수)

• 사용자 정의 변수 사용 시 주의사항

- SET 명령을 이용해 변수를 선언하고 초기화 후 사용
- 변수명은 대소문자 구분하지 않는다.
- 변수를 초기화 하지 않으면 NULL 값으로 초기화 한다.
- 변수의 타입은 초기화 시 치환되는 값에 의해 결정된다.
- 변수를 사용하는 문장에서 선언하지 않는 것이 좋다.

```
SET @변수명 = 값, @변수명 =값;
```

```
SELECT @변수명 = 값, @변수명 := 값;
```

▣ 변수 (사용자 정의 변수)

사용자 정의 변수의 사용

USE hr;

SELECT @low := 4000, @hi := 6000; -- 선언 및 초기화

SELECT @low, @hi; -- 변수의 값 조회

-- 변수 값의 사용

SELECT * FROM employees WHERE salary BETWEEN @low AND @hi
ORDER BY salary;

-- SET @low_sal, @hi_sal;

SELECT @low_sal, @hi_sal;

SET @low_sal := 4500, @hi_sal := 6500;

SELECT @test;

-- 오류. SET으로 변수를 선언할 때에는 초기화도 같이 진행해야 한다.

-- 선언된 변수가 있으면 값을 출력, 없으면 NULL값으로 초기화

-- 변수 선언 및 초기화

-- 변수가 없으므로 NULL

-- 변수의 사용

SELECT * FROM employees WHERE salary BETWEEN @low_sal AND @hi_sal
ORDER BY salary;

▣ 변수 (지역 변수)

-- 지역변수의 사용

DROP PROCEDURE IF EXISTS ifProc; -- 기존의 ifProc 프로시저가 있으면 삭제

CREATE PROCEDURE ifProc()

BEGIN

DECLARE var1 **INT**; -- 지역변수 선언

SET var1 = 100; -- 초기화

IF var1 = 100 **THEN**

SELECT '100입니다.' AS result;

ELSE

SELECT '100이 아닙니다.' AS result;

END IF;

END;

CALL ifProc(); -- PROCEDURE 호출

□ INSERT

• INSERT문

- 테이블에 데이터를 삽입하는 명령문이다.

```
INSERT INTO 테이블명  
[(컬럼명1, 컬럼명2, ...)]  
VALUES  
(값1, 값2, ...);
```

- 기본적으로 한 번에 하나의 행에만 데이터를 입력할 수 있다.
- 테이블 리스트에 있는 컬럼의 개수와 VALUES 절에 있는 값의 개수가 같아야 한다.
- 입력될 값의 크기는 컬럼의 크기보다 크지 않아야 한다.
- 문자와 날짜 타입의 컬럼에는 ' ' 인용부호를 반드시 사용해야 한다.

• AUTO_INCREMENT

- 테이블 생성 시 컬럼의 속성값을 AUTO_INCREMENT로 지정했다면 데이터 삽입 시 값이 자동으로 증가되므로 INSERT 열에서 제외 시킨다.
- auto_increment로 설정된 컬럼에 데이터를 insert 할 때에 NULL로 삽입하면 자동증가 값으로 채워진다.
- DEFAULT 값으로 지정된 데이터는 값을 입력하지 않으면 DEFAULT로 그렇지 않으면 입력된 데이터로 저장된다.

■ INSERT (AUTO_INCREMENT)

• AUTO_INCREMENT

- 테이블 생성 시 컬럼의 속성값을 AUTO_INCREMENT로 지정하면 자동 증가
- 자동 증가하는 값을 확인하거나 수정할 수 있다.

```
USE mydb;
```

```
DROP TABLE mytbl;
```

```
CREATE TABLE mytbl (
  userid int AUTO_INCREMENT PRIMARY KEY
  , name varchar(30)
  , age int
  , join_date DATETIME DEFAULT CURRENT_TIMESTAMP -- 괄호 X
  , today DATE DEFAULT (CURRENT_DATE)           -- 괄호 필요
  , `now` datetime DEFAULT (CURRENT_TIME)        -- 괄호 필요
);
```

```
INSERT INTO mytbl (userid, name, age) VALUES (NULL, '홍길동', 20);
```

```
INSERT INTO mytbl (name, age) VALUES ('손오공', 25);
```

```
SELECT * FROM mytbl;
```

	userid	name	age	join_date	today	now
1	1	홍길동	20	2024-06-14 16:49:58	2024-06-14	2024-06-14 16:49:58
2	2	손오공	25	2024-06-14 16:49:59	2024-06-14	2024-06-14 16:49:59

■ INSERT (AUTO_INCREMENT)

-- AUTO_INCREMENT값 확인하기

```
SELECT last_insert_id();
```

-- 현재 사용한 값 확인 (2)

```
SHOW TABLE status WHERE name = 'mytbl';
```

-- Auto_increment값: 3

-- AUTO_INCREMENT값 수정 (50부터)

```
ALTER TABLE mytbl AUTO_INCREMENT = 50;
```

```
INSERT INTO mytbl (name, age) VALUES ('사오정', 30);
```

```
SELECT * FROM mytbl;
```

	userid	name	age	join_date	today	now
1	1	홍길동	20	2024-06-14 16:49:58	2024-06-14	2024-06-14 16:49:58
2	2	손오공	25	2024-06-14 16:49:59	2024-06-14	2024-06-14 16:49:59
3	50	사오정	30	2024-06-14 16:51:33	2024-06-14	2024-06-14 16:51:33

-- AUTO_INCREMENT값 확인하기

```
SELECT last_insert_id();
```

-- 다음에 사용할 값 확인

-- 테이블 상태 확인

```
SHOW TABLE status WHERE name = 'mytbl';
```

-- Auto_increment값 (다음에 사용할 값: 51)

-- 서버 변수의 증가 값 수정 (10씩 증가하도록 수정)

```
SET @@auto_increment_increment = 10;
```

```
INSERT INTO mytbl (name, age) VALUES ('전우치', 33);
```

```
INSERT INTO mytbl (name, age) VALUES ('삼장법사', 45);
```

```
SELECT * FROM mytbl;
```

	userid	name	age	join_date	today	now
1	1	홍길동	20	2024-06-14 16:49:58	2024-06-14	2024-06-14 16:49:58
2	2	손오공	25	2024-06-14 16:49:59	2024-06-14	2024-06-14 16:49:59
3	50	사오정	30	2024-06-14 16:51:33	2024-06-14	2024-06-14 16:51:33
4	51	전우치	33	2024-06-14 16:52:14	2024-06-14	2024-06-14 16:52:14
5	61	삼장법사	45	2024-06-14 16:52:16	2024-06-14	2024-06-14 16:52:16

□ INSERT

- Table의 속성을 복제하여 테이블 생성하기

```
USE mydb;
```

```
-- 1) hr.employees 테이블과 동일한 구조의 테이블을 mydb.emp1 테이블에 생성한다.
```

```
CREATE TABLE mydb.emp1 LIKE hr.employees;
```

```
-- 2) SELECT 문으로 조회된 결과를 이용해 mydb.emp2 테이블을 생성한다.
```

```
-- 데이터도 복제된다. NULL 관련 제약조건만 동일하게 설정
```

```
CREATE TABLE emp2
```

```
AS
```

```
    SELECT employee_id, first_name, last_name, salary, department_name , email, phone_number
```

```
    FROM hr.employees e INNER JOIN hr.departments d
```

```
    ON (e.department_id = d.department_id)
```

```
WHERE e.commission_pct IS NOT NULL;
```

```
SELECT * FROM emp2;
```

```
DESC emp2;
```

```
-- 3) SELECT 절의 쿼리문에서 반환되는 값이 없으므로 복사되는 데이터는 없이 테이블의 구조만 복사
```

```
CREATE TABLE emp3 AS
```

```
SELECT * FROM hr.employees WHERE 1=2 ;
```

■ INSERT

-- 4) INSERT 문으로 다른 테이블의 데이터 복제

DROP TABLE emp4;

-- 테이블 생성

```
CREATE TABLE emp4 (
    seq int AUTO_INCREMENT PRIMARY KEY,
    first_name varchar(20) NOT NULL,
    last_name varchar(20) NOT NULL
);
```

-- hr.employees에서 필요한 데이터를 추출하여 insert

```
INSERT INTO emp4
    SELECT NULL, first_name, last_name
    FROM hr.employees e ;
```

SELECT * FROM emp4;

	123 seq ▼	ABC first_name ▼	ABC lastt_name ▼
1	1	Steven	King
2	2	Neena	Kochhar
3	3	Lex	De Haan
4	4	Alexander	Hunold
5	5	Bruce	Ernst
6	6	David	Austin
7	7	Valli	Pataballa
8	8	Diana	Lorentz
9	9	Nancy	Greenberg
10	10	Daniel	Faviet
11	11	John	Chen

□ UPDATE

• 데이터의 수정

- 테이블에 입력되어 있던 데이터의 값을 수정하기 위해서는 UPDATE문을 사용하면 된다.

```
UPDATE 테이블명
SET 열이름 = 값, 열이름 = 값
WHERE 조건;
```

- WHERE 절은 생략할 수 있지만 생략 시 테이블의 모든 행 값이 변경되므로 주의해야 한다.
- ROLLBACK 명령을 이용해 작업 내용(TRANSACTION)을 되돌릴 수 있다

```
UPDATE emp2
SET first_name = '제임스';

SELECT * FROM emp2;

ROLLBACK;

UPDATE emp2
SET first_name = '제임스'
WHERE emp2.employee_id = 179;
```

	123 employee_id	ABC first_name	ABC last_name	123 salary	A
1	145	제임스	Russell	14,000	S
2	146	제임스	Partners	13,500	S
3	147	제임스	Errazuriz	12,000	S
4	148	제임스	Cambrault	11,000	S
5	149	제임스	Zlotkey	10,500	S
6	150	제임스	Tucker	10,000	S

31	175	Alyssa	Hutton	8,800	
32	176	Jonathon	Taylor	8,600	
33	177	Jack	Livingston	8,400	
34	179	제임스	Johnson	6,200	

□ DELETE

• 데이터의 삭제

- 테이블에 입력되어 있던 특정 row의 값을 삭제하기 위해서는 DELETE문을 사용하면 된다.

```
DELETE FROM 테이블명
WHERE 조건;
```

- WHERE 절은 생략할 수 있지만 생략 시 테이블의 모든 행 값이 삭제되므로 주의해야 한다.
- ROLLBACK 명령을 이용해 작업 내용(TRANSACTION)을 되돌릴 수 있다

```
DELETE FROM emp2;
```

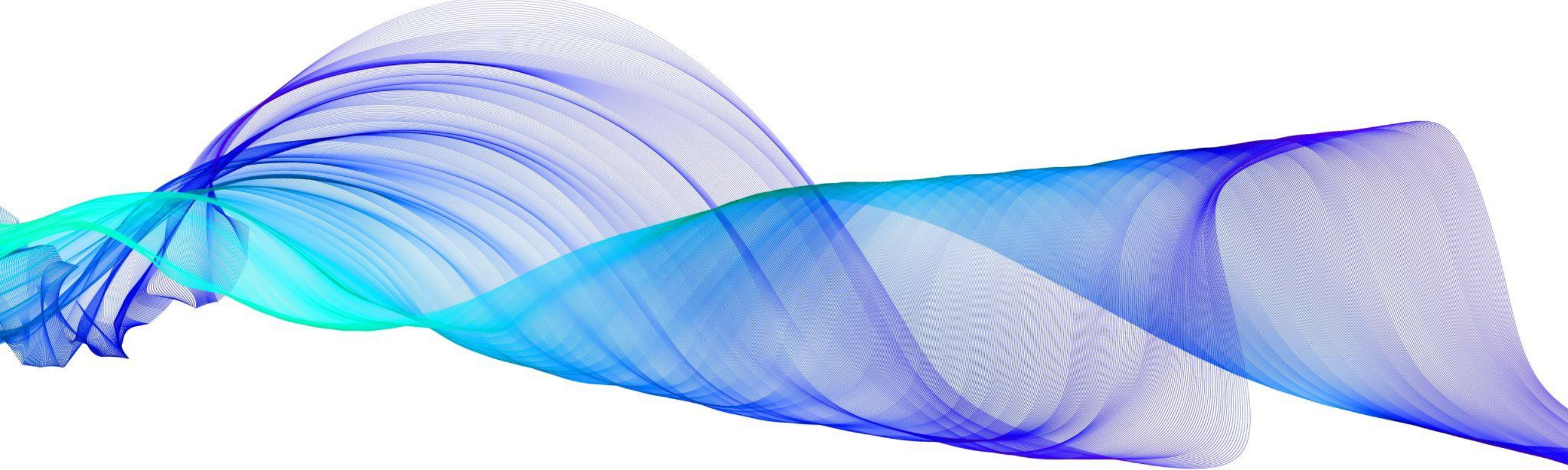
```
SELECT * FROM emp2;
```

```
ROLLBACK;
```

```
DELETE FROM emp2
```

```
WHERE emp2.employee_id = 179;
```

	123 employee_id ▼	ABC first_name ▼	ABC last_name ▼	123 salary ▼
30	174	Ellen	Abel	11,000
31	175	Alyssa	Hutton	8,800
32	176	Jonathon	Taylor	8,600
33	177	Jack	Livingston	8,400



함수

□ 문자열 함수

함수	설명	예
ascii(문자), char(코드)	문자의 코드나 코드의 문자를 반환	SELECT ascii('A'), char(97); -- 65, a
bit_length()	할당된 비트의 크기	SELECT bit_length('abc'), char_length('abc'), length('abc'); -- 24 / 3 / 3
char_length()	문자의 개수	SELECT bit_length('가나다'), char_length('가나다'), length('가나다 '); -- 72 / 3 / 9
length()	할당된 byte 반환	
concat(str1, str2...)	문자열 합치기	SELECT concat('Database', ' mysql ', 'so good~'); -- Database mysql so good~
repeat(str, n);	문자열 반복	SELECT repeat('hello ', 3); -- hello hello hello
reverse(str);	문자열 뒤집기	SELECT reverse('I have a dream'); -- maerd a evah I
locate(str1, str2); locate(str1, str2, n);	str2 문자열 내에 str1의 위치 <u>인덱스는 1부터 시작</u>	SELECT locate('Love', 'Everybody loves the things you do from the way you talk'); -- 11 SELECT locate('you', 'Everybody loves the things you do from the way you talk', 30); -- 48
left(str, n) right(str, n)	문자열 지정한 개수만큼 왼쪽, 오른쪽 뽑음	SELECT left('Everybody loves the things you do from the way you talk', 9), right('Everybody loves the things you do from the way you talk', 4); -- Everybody, talk

□ 문자열 함수

함수	설명	예
insert(str, position, len, s)	str 문자열의 특정 위치에 len 만큼 삭제 후 부분문자열 s를 삽입	SELECT INSERT ('I hate you!', 3, 4, 'love'); -- I love you! -- 3번째 위치에서 4글자 삭제 후 'love' 삽입
substring(str, position, count)	부분 문자열 추출	SELECT substring('Everybody loves the things you do from the way you talk', 11, 4); -- love
lower(str) upper(str)	대소문자 변환	SELECT upper('Everybody loves the things you do from the way you talk'), lower('Everybody loves the things you do from the way you talk'); -- EVERYBODY LOVES THE THINGS YOU DO FROM THE WAY YOU TALK -- everybody loves the things you do from the way you talk
replace(str, s1, s2)	str1 문자열내에서 s1을 s2로 변경	SELECT replace('MSSQL', 'MS', 'My'); -- MySQL
trim(str), ltrim(str), rtrim(str) trim(BOTH s1 FROM str) trim(LEADING s1 FROM str) trim(TRAILING s1 FROM str)	문자열 앞뒤에 포함된 공백이나 지정한 문자열 삭 제 str 문자열내에서 앞이나 뒤 의 지정한 문자열 삭제	SELECT LENGTH(' I have a dream '); -- 18 SELECT LENGTH(trim(' I have a dream ')); -- 14 SELECT LENGTH(ltrim(' I have a dream ')); -- 15 SELECT LENGTH(rtrim(' I have a dream ')); -- 17 SELECT trim(LEADING '!' FROM '!!!! I have a dream!!!'); -- I have a dream!!! SELECT trim(TRAILING '!' FROM '!!!! I have a dream!!!'); -- !!!! I have a dream SELECT trim(BOTH '!' FROM '!!!! I have a dream!!!'); -- I have a dream

▣ 수학 함수

함수	설명	예
abs(n)	숫자 n의 절대값	SELECT ABS(-45.34), ABS(45.23) ; -- 45.34 / 45.34
mod(n1, n2)	n1을 n2로 나눈 나머지 % 연산자와 결과 동일	SELECT MOD(10.5, 3), 10.5%3; -- 1.5 / 1.5
ceiling(n)	소수점 올림	SELECT CEILING(45.153), CEILING(-45.153); -- 46 / -45
floor(n)	소수점 내림	SELECT FLOOR(45.653), FLOOR(-45.653); -- 45 / -46
truncate(n, p)	숫자 n를 소수점 이하 자릿수 p에서 버림.	SELECT TRUNCATE(45.653, 1), TRUNCATE(-45.653, 1); -- 45.6 / -45.6
round(n, p)	숫자 n를 소수점 이하 자릿수 p에서 반올림.	SELECT ROUND(45.653, 1), ROUND(-45.653, 1); -- 45.7 / -45.7
greatest(n1, n2, n3...) least(n1, n2, n3...)	주어진 수 중 제일 큰 수 주어진 수 중 제일 작은 수	SELECT GREATEST(5, 1, 4, 7), LEAST(5, 1, 4, 7); -- 7 / 1
bin(n), hex(n), oct(n)	주어진 수를 2, 16, 8진수로 변환	SELECT bin(12), hex(12), oct(8); -- 1100 / C / 10
conv(n, 진수, 변환 진수)	숫자 n의 진수를 특정 진수 로 변환	SELECT conv('F', 16, 2), conv(15, 10, 16); -- 1111 / F

▣ 수학 함수

함수	설명	예
pi()	원주율	SELECT pi(); -- 3.141593
sqrt(n), pow(n1, n2), power(n1, n2)	제곱근, 제곱 (pow, power는 동일함수)	SELECT sqrt(9), power(2, 3.5), pow(2, 3.5) ; -- 3 / 11.313708498984761
rand()	0~1 미만의 실수 난수	SELECT RAND(), ROUND(RAND()*100, 0); -- 0.88534407586357 / 14
degrees(각도) radians(각도)	각도값을 디그리나 라디안 값으로 변경	SELECT degrees(pi()),radians(180); -- 180 / 3.141592653589793
sin(n) cos(n) tan(n)	삼각함수	SELECT SIN(PI() / 2), COS(PI()), TAN(PI() / 4); -- 1, -1, 1

▣ 날짜/시간 함수

함수	설명	예
now(), sysdate()	현재일의 년 월 일 시 분 초	SELECT now(), sysdate();
curdate(), curtime()	현재일의 날짜, 시간	SELECT curdate(), curtime();
year(날짜), month(날짜), day(날짜)	지정날짜의 년, 월, 일	SELECT year(sysdate()), month(sysdate()), day(sysdate());
date(날짜), time(시간)	지정 날짜의 연월일 지정 날짜의 시분초	SELECT date(sysdate()), time(sysdate()); -- 2024-05-29 / 16:47:40
hour(시간), minute(시간), second(시간)	지정시간의 시, 분, 초	SELECT hour(curtime()) AS '시', minute(curtime()) AS '분', second(curtime()) AS '초'; -- 16 / 48 / 50
datediff(날짜1, 날짜2)	두 날짜 간의 차이	SELECT datediff('1950-6-25', '1945-8-15'); -- 1.775
timediff(시간1, 시간2)	두 시간 간의 차이	SELECT concat('근무한 지 ', timediff(current_time(), ' 08:30:0') , '시간 경과'); -- 근무한 지 08:34:33시간 경과
dayofweek(날짜)	지정 날짜의 요일 (1:일~7:토)	SELECT dayofweek(sysdate()); -- 4 (수요일이라면 4라고 출력)
monthname(날짜)	월 이름	SELECT monthname(sysdate()); -- May
dayofyear(날짜)	1년 중 몇일이 지났는지	SELECT dayofyear(sysdate()); -- 150 (2024-5-29 현재)

▣ 날짜/시간 함수

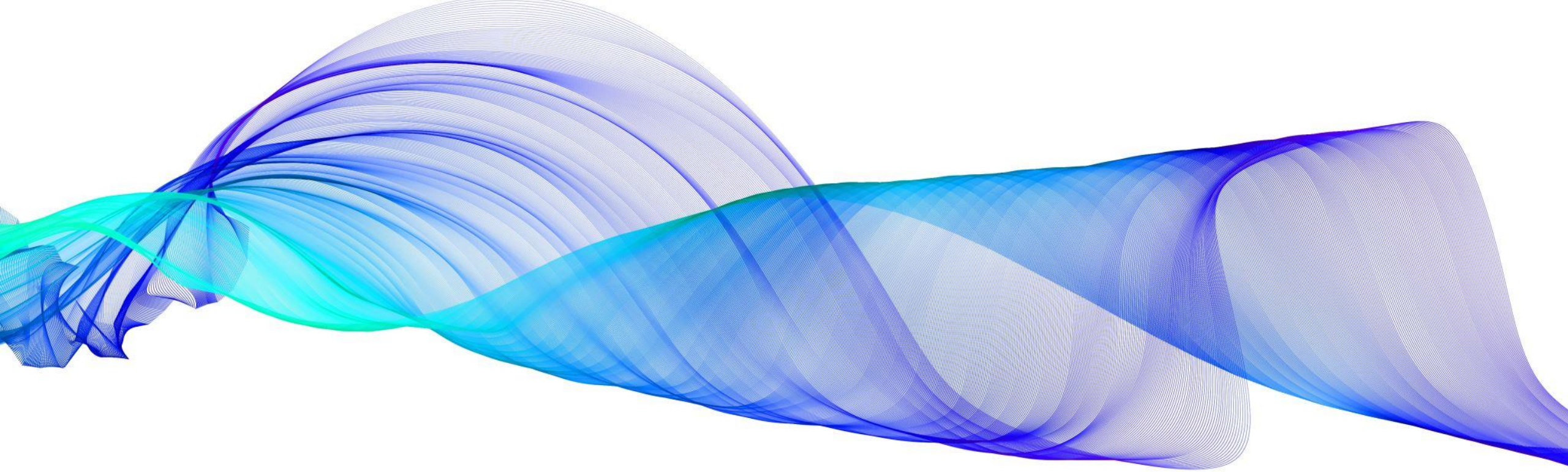
함수	설명	예
last_day(날짜)	주어진 날짜의 마지막 날짜	SELECT last_day(sysdate()); -- 2024-05-31
makedate(연도, 정수)	주어진 연도에서 정수만큼 지난 날짜를 구함	SELECT makedate(2025, 100); -- 2024-01-01에서 100일 지난 날짜
maketime(시, 분, 초)	주어진 시, 분, 초를 이용해 날짜 타입의 데이터를 생성	SELECT maketime(13, 1, 10); -- 13:01:10
period_add(연월, 개월수)	연월로 주어진 날짜에서 특정 개월이 지난 연월을 구함	SELECT period_add(202401, 5); -- 202406
period_diff(연월1, 연월2)	두 날짜 사이의 개월 수 차이	SELECT period_diff(202405, 202312);
quarter(날짜)	주어진 날짜가 몇 분기인지	SELECT quarter(SYSDATE()); -- 2024-05-29일 경우 2분기
time_to_sec(시간)	시간을 초단위로 구함	SELECT time_to_sec('1:10:15'); -- 4,215

▣ 기타 함수

함수	설명	예
if(조건, 참, 거짓)	조건이 참일 경우 참의 작업, 아니면 거짓의 작업 수행	USE sakila; SELECT if(amount > 5, '5보다 많다', '5보다 적다') FROM payment;
ifnull(수식1, 수식2)	수식1이 null이 아니면 수식1을, 아니면 수식2 반환	USE world; SELECT name, region, ifnull(GNPold, '없음') FROM country c WHERE Continent = 'Asia';
nullif(수식1, 수식2)	수식1과 수식2가 같으면 null을 반환 다르면 수식1을 반환	USE world; SELECT name, localname, nullif(name, localname) FROM country c ;
case~when~else~end	연산자의 일종 자바와 같은 언어의 switch 문과 같이 다중분기를 하는 연산자	USE sakila; SELECT DISTINCT CASE film_id WHEN 1 THEN 'Drama' WHEN 2 THEN 'Adventure' WHEN 3 THEN 'Horor' ELSE 'Everything' END FROM film_actor;

▣ 기타 함수

함수	설명	예
user()	현재 접속한 사용자	SELECT user(); -- root@localhost
database()	선택된 데이터베이스	use sakila; SELECT DATABASE(); -- sakila
version()	현재 MySQL 버전	SELECT version(); -- 8.0.36



Grouping

▣ 집계 함수와 Grouping

- Grouping

- 데이터베이스 내의 데이터를 조회할 때 데이터를 특정 조건에 맞는 데이터를 묶어 조회하는 기능
- GROUP BY 절에 정의된 컬럼을 기준으로 ROW를 그룹화 시킨 후 집계 함수를 적용하고, 그룹 당 하나의 결과를 조회.
- 집계 함수는 SELECT 절 또는 having절에 사용할 수 있다.
- group by절에는 그룹 함수를 적용한 컬럼을 정의해야 하며 having절은 그룹의 값을 제한하기 위해 사용

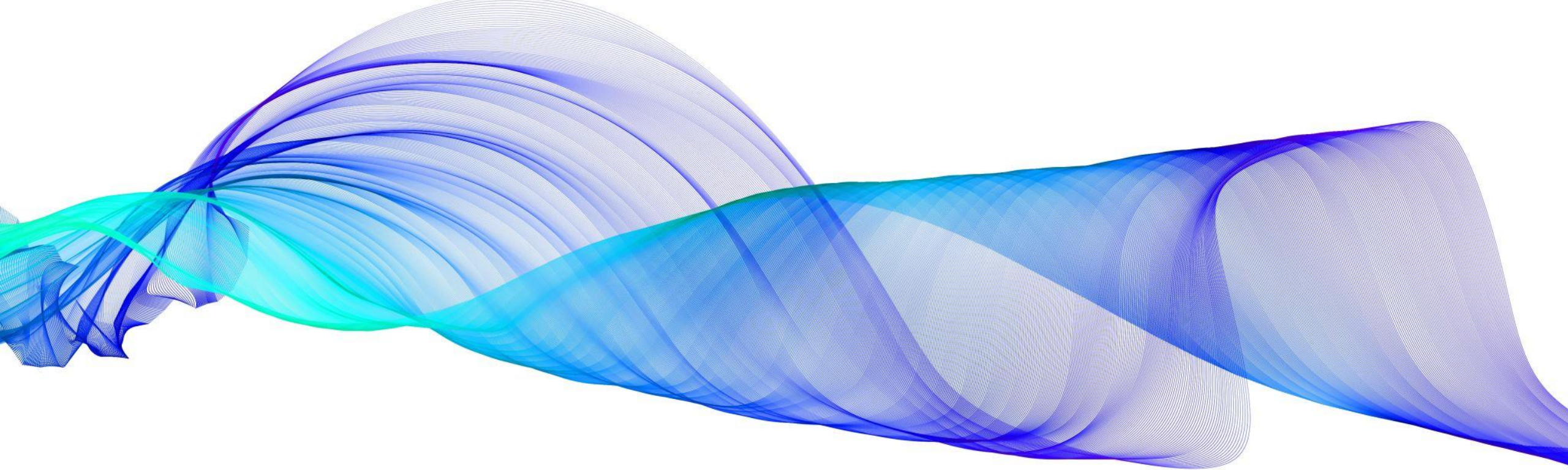
```
SELECT select_expr  
[ FROM 테이블명 ]  
[ WHERE 조건절 ]  
[ GROUP BY [ 컬럼 | expr | position ]  
[ HAVING 그룹조건 ]  
[ ORDER BY expr ];
```

▣ 집계 함수

• 집계 함수란?

- 테이블의 특정 컬럼을 기준으로 각 레코드를 그룹화 시킬 수 있을 경우 그룹핑된 컬럼을 기준으로 수행되는 함수를 집계함수, 혹은 그룹행 함수라고 한다.
- 컬럼을 그룹화하고 그에 대한 조건이 필요할 경우 select 문의 having 절을 사용한다.

함수	설명	예
count(컬럼명)	NULL 값이 아닌 row 개수를 구한다.	SELECT count(*) FROM payment p ;
sum(컬럼명)	컬럼의 합계를 구한다.	SELECT sum(amount), avg(amount) FROM payment; -- 67406.56 4.201356
avg(컬럼명)	각각의 그룹 안에서 필드명의 평균값을 구한다.	
max(컬럼명)	최대값	SELECT min(amount), max(amount) FROM payment; -- 0.00 11.99
min(컬럼명)	최소값	



Join

□ JOIN

• Join이란?

- 관계형 데이터베이스의 테이블은 데이터의 중복과, 메모리 낭비를 피하고 데이터의 무결성을 위해 테이블을 관련된 컬럼들을 분리하여 저장한다.
- 분리된 여러 테이블 내의 컬럼들을 하나의 쿼리문을 통해 한번에 조회하는 것을 JOIN이라고 한다.
- 주로 Primary-Key와 Foreign-Key의 관계를 가진 컬럼을 소유하고 있는 테이블을 통한 검색 시 사용
- 어떤 경우에는 이러한 PK, FK의 관계가 없어도 논리적인 값들의 연관만으로 JOIN이 성립 가능하다.
- JOIN은 관계형 데이터베이스의 가장 큰 장점이면서 대표적인 핵심 기능이다.
- JOIN을 하고자 하는 테이블이 여러 개일 경우 FROM 절에 해당 테이블들을 나열한다.
- FROM 절에 여러 테이블이 나열되더라도 SQL에서 데이터를 처리할 때는 단 두 개의 집합 간에만 조인이 일어난다.

▣ JOIN의 종류 (Cross Join)

- Cross Join(상호조인)

- 한쪽 테이블의 모든 행들과 다른 쪽 테이블의 모든 행을 조인.
- Cartesian Product (카르테시안 곱)이라고도 한다.
- Join을 위한 where 절이나 on 절 등 조인조건에 관련된 구문이 없다.

- Cross Join의 사용

```
USE hr;  
  
-- 1) Cross Join  
SELECT *  
FROM employees CROSS JOIN departments;  
  
SELECT *  
FROM employees, departments;
```

▣ JOIN의 종류 (Inner Join)

• Inner Join(내부조인)

- 가장 많이 사용되는 조인 방식.
- PK와 FK의 연관 관계를 갖는 둘 혹은 그 이상이 테이블들 간에 같은 PK와 FK를 갖는 테이블들을 결합하여 조회
- FK 컬럼의 값이 null인 경우 조인 대상에서 제외된다.
- 조인하고자 하는 두 테이블의 PK, FK의 교집합이라고 보면 된다.
- INNER 키워드 생략 가능

• Inner Join의 사용

```
USE hr;
```

```
-- 2) Inner Join
```

```
SELECT *
```

```
FROM employees e INNER JOIN departments d
```

```
ON e.department_id = d.department_id;
```

-- 두 테이블의 조인 조건을 ON 절에 기재 (INNER)

```
SELECT *
```

```
FROM employees e, departments d
```

```
WHERE e.department_id = d.department_id;
```

-- 두 테이블의 조인 조건을 where 절에 기재

JOIN의 종류 (Inner Join)

- Inner Join의 사용 예

```
-- 급여가 200을 초과하는 직원의 이름과 급여, 부서명을 조회하시오
```

```
SELECT e.first_name, e.salary, d.department_name  
FROM employees e INNER JOIN departments d  
ON  
e.department_id = d.department_id  
AND  
e.salary > 200;
```

```
-- 급여가 200을 초과하는 직원의 이름과 급여, 부서명, 근무하는 도시를 조회하시오  
-- 세 개의 테이블이 필요 (employees, departments, locations)
```

```
SELECT e.first_name, e.salary, d.department_name, l.city  
FROM employees e  
INNER JOIN departments d  
ON e.department_id = d.department_id  
INNER JOIN  
locations l  
ON d.location_id = l.location_id  
AND  
e.salary > 200;
```

▣ JOIN의 종류 (Inner Join)

- Inner Join의 사용 예

```
-- 급여가 200을 초과하는 직원의 이름과 급여, 부서명을 조회하시오
```

```
-- 첫 번째 방법
```

```
SELECT e.first_name, e.salary, d.department_name  
FROM employees e INNER JOIN departments d  
ON  
    department_id = d.department_id  
AND  
    e.salary > 200;
```

```
-- 두 번째 방법
```

```
SELECT e.first_name, e.salary, d.department_name  
FROM employees e, departments d  
WHERE  
    e.department_id = d.department_id  
AND  
    e.salary > 200;
```

▣ JOIN의 종류 (Inner Join)

- Inner Join의 사용 예

```
-- 급여가 200을 초과하는 직원의 이름과 급여, 부서명, 근무하는 도시를 조회하시오
-- 세 개의 테이블이 필요 (employees, departments, locations)
-- 첫 번째 방법
SELECT e.first_name, e.salary, d.department_name, l.city
FROM employees e INNER JOIN departments d
    ON e.department_id = d.department_id
    INNER JOIN locations l
    ON d.location_id = l.location_id
    AND
        e.salary > 200;

-- 두 번째 방법
SELECT e.first_name, e.salary, d.department_name, l.city
FROM employees e, departments d, locations l
WHERE
    e.department_id = d.department_id
    AND
    d.location_id = l.location_id
    AND
    e.salary > 200;
```

▣ JOIN의 종류 (Outer Join)

• Outer Join(내부조인)

- 기본적인 조회방식은 Inner Join과 유사하며 조인 조건에서 동일한 값이 없는 행도 조회하고자 할 때 사용되는 조인 방식.
- FK의 역할을 하는 컬럼의 값이 null인 경우에도 결과를 조회할 수 있다.
- Outer Join은 기준이 되는 테이블이 어느 쪽에 위치 하느냐에 따라 LEFT/RIGHT/FULL 방식이 있다.
- LEFT/RIGHT는 생략할 수 없으나 OUTER 키워드는 생략가능
- left, right 테이블의 조인 컬럼의 이름이 같을 경우 ON절 대신 Using 절을 사용할 수 있다.
- Left를 쓸 경우 왼쪽 테이블 공통 컬럼의 Null Data는 모두 조회, 오른쪽 테이블의 데이터는 출력하지 않음
- Full을 쓸 경우 왼쪽과 오른쪽 양쪽 테이블의 Null Data를 모두 출력함

• Outer Join 형식

```
-- Outer Join 형식-1
SELECT 열목록
FROM left-table <LEFT | RIGHT | FULL> [OUTER] JOIN right-table
ON left-column = right-column;
```

```
-- Outer Join 형식-2
SELECT 열목록
FROM left-table <LEFT | RIGHT | FULL> [OUTER] JOIN right-table
USING (column명);
```

▣ JOIN의 종류 (Outer Join)

- Outer Join의 사용 예

```
-- 왼쪽 테이블을 기준으로 Outer Join. 왼쪽 테이블의 Null 데이터는 모두 조회
```

```
-- Outer 키워드 생략 가능
```

```
SELECT e.first_name, e.salary, d.department_name  
FROM employees e LEFT OUTER JOIN departments d  
ON e.department_id = d.department_id; --
```

```
-- ON 대신 USING 사용
```

```
-- USING을 사용할 때에든 두 테이블의 공통 컬럼의 이름이 동일해야한다.
```

```
SELECT e.first_name, e.salary, d.department_name  
FROM employees e LEFT JOIN departments d  
USING (department_id);
```

▣ JOIN의 종류 (Outer Join)

- Outer Join의 사용 예

```
-- [문제] 부서명과 부서가 위치한 도시와 해당 도시가 있는 나라를 조회하시오
-- 단, 테이블에 존재하는 모든 국가와 도시명이 전부 출력되어야 하며 USING 절을 사용하시오.
-- 필요한 테이블(departments, locations, countries)
SELECT c.country_name, l.city, d.department_name
FROM countries c
      LEFT JOIN locations l
      USING (country_id)
      LEFT JOIN departments d
      USING (location_id);

-- [문제] 부서명과 부서가 위치한 도시와 해당 도시가 있는 나라를 조회하시오.
-- 부서가 없는 도시와 국가는 조회대상에서 제외할 것 (ON절 사용)
-- 필요한 테이블(departments, locations, countries)
SELECT c.country_name, l.city, d.department_name
FROM countries c
      RIGHT JOIN locations l
      ON c.country_id = l.country_id
      RIGHT JOIN departments d
      ON l.location_id = d.location_id ;
```

▣ JOIN의 종류 (Self Join)

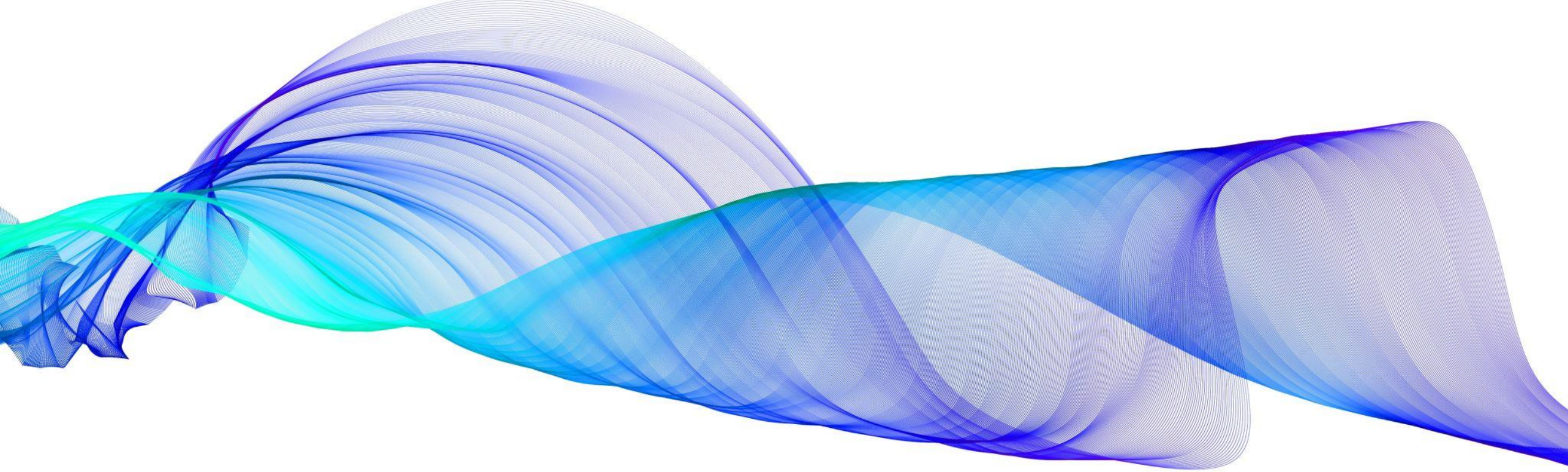
- Self Join (자체 조인)

- 하나의 테이블 내의 특정 컬럼이 자신이 속한 테이블의 PK를 참조하는 형태일 때 사용하는 조인방식이다.
- 즉, 하나의 테이블 내에 PK와 그 PK를 참조하는 FK가 함께 있는 경우에 사용된다.
- 사용 방식은 Inner Join/Outer Join과 같으며

- Self Join 사용 예

```
-- (4) Self Join  
-- [문제] 사원번호, 이름, 팀장의 이름을 출력하고, 팀장이 없는 직원은 "없음"이라고 출력하시오.
```

```
SELECT  
    e1.employee_id,  
    concat(e1.first_name, ' ', e1.last_name) AS "직원명",  
    ifnull(concat(e2.first_name, ' ', e2.last_name), "없음" ) AS "팀장명"  
FROM employees e1  
LEFT JOIN employees e2  
ON e1.manager_id = e2.employee_id;
```



Subquery

□ Subquery

- 개념

- Query 문장 안에 또 다른 Query 문장이 포함된 쿼리

- 용어

- ① Main Query

- 실제로 실행될 SQL 문을 가지고 있다
 - Subquery에 의해 실행된 SQL문의 결과에 의해 Main Query가 실행된다.

- ② Subquery

- Main Query 안쪽에 위치한 Query문
 - 연산자의 오른쪽에 위치하며 괄호로 묶어 사용된다.
 - 질의가 지정되지 않은 값을 근거로 할 때 유용하다.
 - 대부분 Subquery가 먼저 실행되고, 그 결과가 Main Query에 전달되어 실행된다.
 - Subquery에서는 ORDER BY절은 사용할 수 없다. (SELECT/DELETE/UPDATE 문장의 FROM 절에 사용하는 경우 제외)

□ Subquery

- Subquery의 사용 위치
 - SELECT 절
 - WHERE 절
 - FROM 절
 - HAVING 절
 - INSERT 문의 INTO절
 - UPDATE 문의 SET절

▣ Subquery의 종류

- Sub Query의 종류

- 일반적인 서브 쿼리

- 서브 쿼리의 결과를 하나의 값처럼 사용.

- ① 단일 행 서브 쿼리 (Single Row Subquery) : 쿼리 결과가 하나의 행으로 반환되는 서브 쿼리

- ② 다중 행 서브 쿼리 (Multi Row Subquery) : 쿼리 결과가 여러 개의 행으로 반환되는 서브 쿼리

- ③ 다중칼럼 서브 쿼리 (Multi Column Subquery) : 쿼리 결과가 여러 개의 칼럼으로 반환되는 서브 쿼리

- 인라인 뷰(Inline View)

- 다중 행, 다중 컬럼을 가진 테이블의 형태(= View)로 반환되는 서브 쿼리로, 이렇게 반환된 쿼리의 결과는 테이블처럼 사용 가능
 - Select 문의 From 절에서 사용 또는 JOIN 할 테이블을 반환할 때 사용

- 스칼라 서브 쿼리(Scalar Subquery)

- 쿼리 결과를 하나의 컬럼처럼 사용하는 서브 쿼리

- 상호연관 서브 쿼리(Correlative Subquery)

- Main Query의 값을 Subquery에서 사용
 - Main Query의 결과로 Subquery가 실행되고, Subquery의 결과를 다시 Main Query가 사용하는 서로 연관된 형태의 쿼리

▣ Subquery (단일 행 서브 쿼리)

1) 단일 행 서브 쿼리

- 서브 쿼리의 실행 결과가 하나의 행이나 하나의 컬럼을 반환해 주는 경우
- 비교연산자 사용한다. (=, <, >, <=, >=)
- 서브 쿼리의 결과가 여러 개를 반환하는 경우 에러가 발생한다.
- 서브 쿼리에 의해 반환되는 되는 결과의 컬럼 수와 메인 쿼리에서 비교되는 컬럼 수가 일치하지 않으면 에러

▣ Subquery (다중 행 서브 쿼리 / 다중 컬럼 서브 쿼리)

2) 다중 행 서브 쿼리

- 서브 쿼리의 실행 결과가 여러 개의 행을 반환
- 반환되는 결과가 여러 개의 행이기 때문에 다중 행을 비교할 수 있는 연산자를 사용해야 한다.

① IN 연산자

- 서브 쿼리에 의한 결과에 해당하는 행이 있으면 출력한다.

② ANY 연산자

- 서브 쿼리에 의한 결과의 값에 하나 이상 만족하는 행이 있으면 출력한다.
- = ANY, > ANY, < ANY, <= ANY, >=ANY, != ANY 가 있다.
- = ANY는 IN 연산자와 동일함

③ ALL 연산자

- 서브 쿼리에 의한 결과의 값이 조건을 만족하면 출력한다.
- < ALL : 결과 중 가장 작은 값보다 작은 조건의 행
- > ALL : 결과 중 가장 큰 값보다 큰 조건의 행

▣ Subquery (다중 행 서브 쿼리 / 다중 컬럼 서브 쿼리)

④ EXISTS 연산자

- 서브 쿼리에 의한 결과에 반환되는 행이 존재하는지 여부를 확인
- sub query가 하나 이상의 행을 반환하면 true를, 행을 반환하지 않으면 false를 반환하는 bool 연산자다. 주로 SELECT, INSERT, UPDATE, DELETE 문의 WHERE 절과 함께 사용되며 다른 테이블에 관련 레코드의 존재하는지 여부를 판단하여 필터링 한다.
- 한 테이블이 다른 테이블과 외래키(FK)와 같은 관계가 있을 때 유용
- 조건에 해당하는 ROW의 존재 유무와 이후 더 수행하지 않음 (지연 평가 원리 이기 때문에 성능이 좋다)
- 일반적으로 SELECT절까지 가지 않기에 IN에 비해 속도나 성능면에서 더 좋음
- 반대로 조건에 맞지 않는 ROW만 추출하고 싶으면 NOT EXISTS
- 쿼리 순서 : 메인 쿼리 → EXISTS 쿼리

3) 다중 컬럼 서브 쿼리

- 서브 쿼리의 실행 결과가 여러 개의 컬럼을 반환
- 다중 행 서브 쿼리와 같이 연산자와 함께 사용한다.

▣ Subquery (다중 행 서브 쿼리 / 다중 컬럼 서브 쿼리)

• IN 연산자의 사용

- where 조건식에서 OR 연산으로 여러 개의 조건들 중 하나의 값과 같을 경우를 조회하는 것과 유사하다.
- 서브 쿼리의 조회 결과가 여러 개일 때 그 중 하나의 결과에 만족하는 메인 쿼리를 출력한다.

```
-- [문제] MySQL에서 제공하는 world database 내의 테이블을 확인한 후  
-- 전체 인구 Percentage 100%가 공식언어로 'Spanish'를 사용하는 나라의 이름과 인구수를  
-- 조회하는 코드를 서브 쿼리를 이용해 작성 하시오.
```

```
USE world;
```

```
SELECT name, Population  
FROM country c  
WHERE code IN (SELECT CountryCode  
               FROM countrylanguage  
               WHERE  
                 Percentage = 100  
                 AND  
                 `Language` = 'Spanish');
```

▣ Subquery (다중 행 서브 쿼리 / 다중 컬럼 서브 쿼리)

• ANY 연산자의 사용

- 서브 쿼리의 결과가 여러 개가 조회될 경우 조회된 모든 값들 어느 하나라도 만족되면 결과가 조회
- 비교 연산자와 ANY 키워드가 함께 사용된다.
- = ANY는 OR 연산과 결과가 동일하다.

```
-- ANY 연산자의 사용
-- [문제] 'ST_MAN'이라는 직군이 받는 급여보다 적은 급여를 받는 직원의 이름과 직업을 조회하시오.
-- 'ST_MAN' 직군의 급여는 5종류가 있으며 그 중 가장 적은 급여는 5,800, 가장 많은 급여는 8,200이다.
-- 이 급여들 보다 적은 급여를 받는 것이므로 OR 연산과 유사하며, 8,200보다 적은 급여를 받는 사람들의 명단을 출력해야 한다.
```

```
USE hr;
```

```
SELECT FIRST_NAME, SALARY
FROM EMPLOYEES
WHERE SALARY < ANY (SELECT SALARY FROM EMPLOYEES
                    WHERE JOB_ID = 'ST_MAN'
                    ORDER BY SALARY);
```

Subquery (다중 행 서브 쿼리 / 다중 컬럼 서브 쿼리)

-- [문제] job_title이 "Manager"인 직원과 동일한 급여를 받는 사원의 사번, 이름, JOB_ID, 급여를 급여순으로 조회하시오.

```
SELECT employee_id, first_name, job_id, salary
FROM employees
WHERE salary = ANY (SELECT e.salary
                     FROM employees e JOIN jobs j
                     USING (job_id)
                     WHERE job_title LIKE '%Manager')
ORDER BY salary;
```

	123 employee_id ▼	ABC first_name ▼	ABC job_id ▼	123 salary ▼
1	124	Kevin	ST_MAN	5,800
2	123	Shanta	ST_MAN	6,500
3	203	Susan	HR_REP	6,500
4	122	Payam	ST_MAN	7,900
5	120	Matthew	ST_MAN	8,000
6	153	Christopher	SA_REP	8,000
7	159	Lindsey	SA_REP	8,000
8	110	John	FI_ACCOUNT	8,200
9	121	Adam	ST_MAN	8,200
10	149	Eleni	SA_MAN	10,500
11	162	Clara	SA_REP	10,500
12	114	Den	PU_MAN	11,000
13	148	Gerald	SA_MAN	11,000
14	174	Ellen	SA_REP	11,000

▣ Subquery (다중 행 서브 쿼리 / 다중 컬럼 서브 쿼리)

- ALL 연산자의 사용

- where 조건식에서 AND 연산으로 여러 개의 조건을 조회하는 것과 유사하다.

```
-- ALL 연산자의 사용
-- [문제] 'ST_MAN'이라는 직군이 받는 급여보다 적은 급여를 받는 직원의 이름과 직업을 조회하시오.
-- 'ST_MAN' 직군의 급여는 5종류가 있으며 그 중 가장 적은 급여는 5,800, 가장 많은 급여는 8,200이다.
-- 이 급여들 보다 적은 급여를 받는 것이므로 OR 연산과 유사하며, 5,800보다 적은 급여를 받는 사람들의 명단을 출력해야 한다.
```

```
USE hr;
```

```
SELECT FIRST_NAME, SALARY
FROM EMPLOYEES
WHERE SALARY < ALL (SELECT SALARY FROM EMPLOYEES
                    WHERE JOB_ID = 'ST_MAN'
                    ORDER BY SALARY);
```

Subquery (다중 행 서브 쿼리 / 다중 컬럼 서브 쿼리)

-- [문제] job_title이 "Manager"인 직원과 동일한 급여를 받는 사원의 사번, 이름, JOB_ID, 급여를 급여순으로 조회하시오.

```
SELECT employee_id, first_name, job_id, salary
FROM employees
WHERE salary = ALL (SELECT e.salary
                    FROM employees e JOIN jobs j
                    USING (job_id)
                    WHERE job_title LIKE '%Manager')
ORDER BY salary;
```

	123 employee_id ▼	ABC first_name ▼	ABC job_id ▼	123 salary ▼
1	124	Kevin	ST_MAN	5,800
2	123	Shanta	ST_MAN	6,500
3	203	Susan	HR_REP	6,500
4	122	Payam	ST_MAN	7,900
5	120	Matthew	ST_MAN	8,000
6	153	Christopher	SA_REP	8,000
7	159	Lindsey	SA_REP	8,000
8	110	John	FI_ACCOUNT	8,200
9	121	Adam	ST_MAN	8,200
10	149	Eleni	SA_MAN	10,500
11	162	Clara	SA_REP	10,500
12	114	Den	PU_MAN	11,000
13	148	Gerald	SA_MAN	11,000
14	174	Ellen	SA_REP	11,000

▣ Scalar Subquery

• Scalar Subquery

- SELECT 명령문의 컬럼 위치에서 서브 쿼리를 사용하여 서브 쿼리의 결과가 마치 하나의 컬럼과 같이 사용되는 쿼리를 말한다.
- 스칼라 서브 쿼리는 컬럼위치에 서브 쿼리가 사용되므로 반드시 하나의 ROW만 반환되어야 한다.
- 스칼라 서브 쿼리보다 JOIN을 사용하는 것이 더 바람직하다 (성능 문제)

-- [문제] 이름이 'Bruce', 'Daniel'이라는 직원의 성을 조회하시오

SELECT

(SELECT LAST_NAME FROM EMPLOYEES WHERE FIRST_NAME = 'Bruce') AS "Bruce의 성",

(SELECT LAST_NAME FROM EMPLOYEES WHERE FIRST_NAME = 'Daniel') AS "Daniel의 성"

;

	ABC Bruce의 성 ▼	ABC Daniel의 성 ▼
1	Ernst	Faviet

▣ Scalar Subquery

-- [문제] 사원명, 부서번호, 급여, 부서명을 조회하시오. (결과는 동일하다)

-- 1) Scalar Query 및 Correlative Query(상호 연관 쿼리)의 사용

```
SELECT e.first_name, e.department_id, e.salary,
       (SELECT department_name FROM departments d WHERE d.department_id = e.department_id) AS "부서명"
FROM employees e;
```

-- 2) Outer Join 사용

```
SELECT e.first_name, e.department_id, e.salary, d.department_name
FROM employees e LEFT OUTER JOIN departments d
USING (department_id);
```

	ABC first_name ▼	123 department_id ▼	123 salary ▼	ABC 부서명 ▼
1	Steven	90 ↗	24,000	Executive
2	Neena	90 ↗	17,000	Executive
3	Lex	90 ↗	17,000	Executive
4	Alexander	60 ↗	9,000	IT
5	Bruce	60 ↗	6,000	IT
6	David	60 ↗	4,800	IT
7	Valli	60 ↗	4,800	IT

□ Inline View

- **Inline View란?**

- FROM절 괄호 안에 기술된 SELECT문(Sub Query)을 INLINE VIEW라고 한다.
- 상호연관 Sub Query 에서 똑같은 절차를 계속 반복하는 것은 비효율적이므로 INLINE VIEW를 이용한다고 했었다.
- INLINE VIEW라는 가상의 테이블을 만들어서 다른 테이블과 조인하여 사용한다.
- INLINE VIEW에서 그룹함수 같이 긴 표현식으로 나타나는 컬럼 이름들은 별칭을 지정해야 하며. 만약 별칭을 지정하지 않으면, 표현식이 컬럼 이름인지 함수 값인지 모호하므로 오류가 발생한다.

□ Inline View

-- FROM 절에 사용된 서브 쿼리 (Inline View라고도 한다.)

SET @ROWNUM:=0; -- MySQL에는 오라클의 rownum의 개념이 없기 때문에 변수에 값을 대입하여 사용

SELECT @ROWNUM := @ROWNUM+1 AS ROWNUM, TBL.*
FROM

(SELECT EMPLOYEE_ID, FIRST_NAME, SALARY, DEPARTMENT_ID
FROM EMPLOYEES
WHERE DEPARTMENT_ID = 80) TBL

WHERE TBL.SALARY > 8000;

	123 ROWNUM	ABC EMPLOYEE_ID	123 FIRST_NAME	123 SALARY	123 DEPARTMENT_ID
1	1	145	John	14,000	80
2	2	146	Karen	13,500	80
3	3	147	Alberto	12,000	80
4	4	148	Gerald	11,000	80
5	5	149	Eleni	10,500	80
6	6	150	Peter	10,000	80
7	7	151	David	9,500	80
8	8	152	Peter	9,000	80
9	9	156	Janette	10,000	80
10	10	157	Patrick	9,500	80
11	11	158	Allan	9,000	80
12	12	162	Clara	10,500	80
13	13	163	Danielle	9,500	80
14	14	168	Lisa	11,500	80

▣ 상호연관 Subquery (Correlative Subquery)

• 상호연관 서브 쿼리

- Main Query 가 사용한 결과값을 Sub Query가 이용하여 결과를 조회하는 쿼리문이다.
- Main Query(outer query)의 한 개 행을 먼저 실행하여 그 결과를 조회한 후 해당 행이 서브쿼리의 조건을 충족하는지 확인하고 select를 진행한다.

• 상호연관 서브 쿼리 실행 흐름

- ① Main Query에서 첫 번째 행을 실행하여 그 결과(후보행)를 Sub Query에 전달.
 - ② Sub Query에서는 전달된 행의 값을 사용해서 Sub Query를 수행.
 - ③ Sub Query 의 실행 결과값을 전달된 행과 비교했을 때 참이면 별도의 메모리에 전달된 행 정보를 저장, 거짓이면 drop
 - ④ Main Query에서 다음 행을 이용하여 2, 3, 4번을 반복적으로 수행
- Sub Query 의 WHERE 절에 들어가는 값이 상수가 아니라 미지수(or 후보행 값)일 때, 미지수를 설명해주는 Main Query를 먼저 실행하고 그 결과의 저장할 것인지 여부를 결정하고 처리하는 작업을 Sub Query 가 끝날 때까지 반복 수행한다.
 - Sub Query가 끝날 때까지 반복 수행한다는 점은 상호연관 Sub Query의 문제점이기도 하다. 상호연관 Sub Query 는 동일한 동작을 " 반복 " 하기 때문에 똑같은 쿼리를 던지고 그에 대한 결과를 얻고 또다시 똑같은 쿼리를 던지고 결과를 받고 하는 식의 과정을 계속해서 반복한다.
즉, 똑같은 후보행 값이 입력되더라도 무조건 서브쿼리는 수행해야 한다.
 - 그러므로 보다 복잡한 쿼리문을 실행 해야 할 때 똑같은 절차를 계속 반복하는 것은 비효율적일 수 있다. 따라서 이 문제점을 해결할 수 있는 방법으로 INLINE VIEW라는 가상 테이블을 이용할 수 있다.

▣ 상호연관 Subquery와 Inline View

-- [문제] 각 부서별로 해당 부서의 급여 평균 미만의 급여를 수령하는 직원의 명단을 출력하시오.

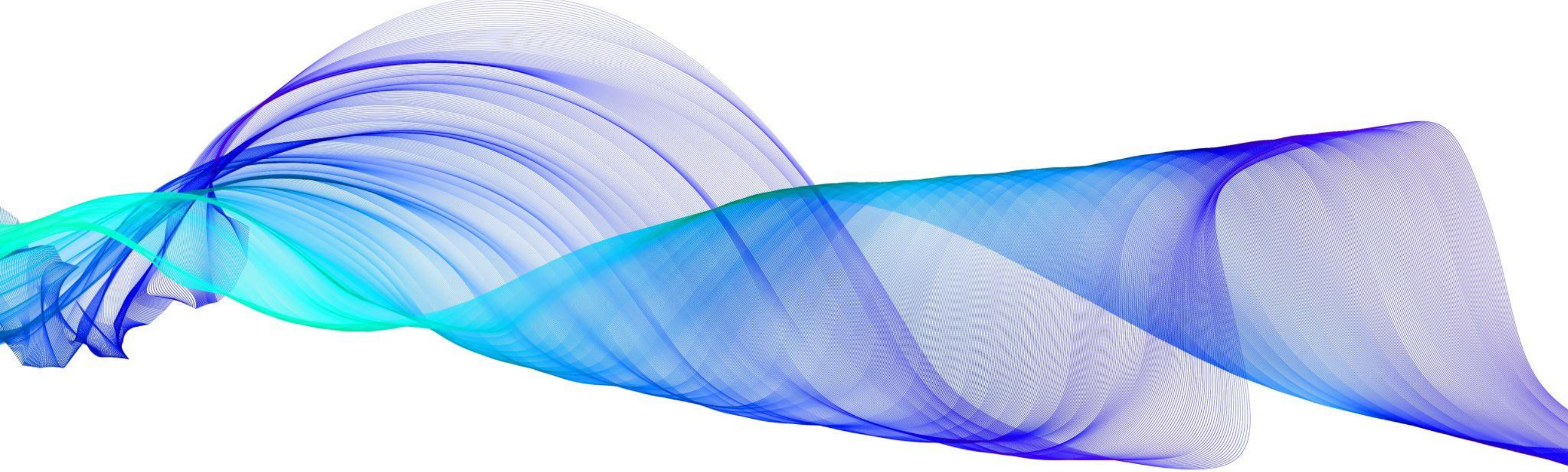
-- (부서번호, 사원번호, 이름, 급여, 부서별 평균급여)

-- 1) 상호 연관 쿼리의 결과

```
SELECT department_id, employee_id, first_name, salary
FROM employees e
WHERE salary < (SELECT avg(salary)
                FROM employees
                WHERE department_id = e.department_id)
ORDER BY 1, 2;
```

▣ 상호연관 Subquery와 Inline View

```
-- [문제] 각 부서별로 해당 부서의 급여 평균 미만의 급여를 수령하는 직원의 명단을 출력하시오.  
-- (부서번호, 사원번호, 이름, 급여, 부서별 평균급여)  
  
-- 2) inline view의 처리 결과  
SELECT e2.department_id, e2.employee_id, e2.first_name, e2.salary, round(e1.avgsal) 부서평균  
FROM (SELECT department_id as dept_id, avg(salary) as avgsal  
      FROM employees  
      GROUP BY department_id) e1, employees e2  
WHERE e1.dept_id = e2.department_id  
AND e2.salary < e1.avgsal  
ORDER BY 1;
```



DDL(Data Definition Language)

▣ DDL (Data Definition Language)

- DDL이란?

- 주로 DBA(Data Base Administrator)가 사용하거나, 스키마 보유한 자가 실행하는 데이터 베이스를 정의하는 언어.
- schema, table, view, index등 Object를 생성, 수정, 삭제하는 등의 데이터 관리를 위한 전체 골격을 결정하는 역할을 담당한 언어.
- 종류
 - ① CREATE : 생성
 - ② ALTER : 수정
 - ③ DROP : 삭제
 - ④ TRUNCATE : 초기화

□ CREATE - Schema

- [예]

-- 1. 현재 생성된 데이터베이스의 종류를 나열

SHOW DATABASES;

	ABC Database ▼
1	information_schema
2	mydb
3	mysql
4	performance_schema
5	sakila
6	sys
7	world

-- 2. 한글세트를 저장할 수 있는 데이터베이스 생성

CREATE DATABASE 'hr' DEFAULT CHARACTER SET UTF8MB3;

-- 3. 데이터베이스 삭제

DROP DATABASE 'hr';

-- 4. 데이터베이스 사용

USE 'hr';

▣ CREATE - Schema

• INFORMATION_SCHEMA

- DB 서버에 존재하는 메타 정보(테이블, 칼럼, 인덱스 등의 스키마 정보)를 모아둔 Database이다.
- INFORMATION_SCHEMA 데이터베이스 내의 메타 정보를 담고 있는 테이블은 조회만 가능하고 사용자가 내용을 직접 수정할 수 없다.
- 이 테이블은 레코드가 있는 것이 아니라, SQL을 이용해 조회할 때마다 메타 정보를 MySQL 서버의 메모리에서 가져와서 출력한다. 이런 이유로 한꺼번에 많은 테이블을 조회할 경우 시간이 많이 걸릴 수도 있다.
- 특정 스키마 내의 제약조건 정보 등을 조회할 때 사용한다
- INFORMATION_SCHEMA 데이터베이스 내 80여개의 테이블이 존재한다.

※ 메타 데이터(Metadata)란 의사 결정을 위한 실제 데이터가 아니라 실제 데이터를 관리하기 위한 데이터로, 데이터베이스 또는 테이블의 이름, 컬럼의 데이터 타입, 또는 접근 권한과 같은 것을 말한다. 경우에 따라서 데이터 사전(**data** dictionary) 과 시스템 카탈로그(**system catalog**)로 불린다.

■ CREATE - Schema

• INFORMATION_SCHEMA 의 사용

USE information_schema;

SHOW tables;

	ABC Tables_in_information_schema
1	ADMINISTRABLE_ROLE_AUTHORIZATIONS
2	APPLICABLE_ROLES
3	CHARACTER_SETS
4	CHECK_CONSTRAINTS
5	COLLATION_CHARACTER_SET_APPLICABILITY
6	COLLATIONS
7	COLUMN_PRIVILEGES
8	COLUMN_STATISTICS
9	COLUMNS
10	COLUMNS_EXTENSIONS
11	ENABLED_ROLES
12	ENGINES
13	EVENTS
14	FILES
15	INNODB_BUFFER_PAGE
16	INNODB_BUFFER_PAGE_LRU
17	INNODB_BUFFER_POOL_STATS
18	INNODB_CACHED_INDEXES
19	INNODB_CMP
20	INNODB_CMP_PER_INDEX
21	INNODB_CMP_PER_INDEX_RESET

■ CREATE - Schema

- INFORMATION_SCHEMA 내의 테이블을 이용해 제약조건 확인

< 제약 조건의 확인 >

-- 제약조건 확인 (check, primary key, foreign key, unique)

```
SELECT
    TABLE_NAME,
    CONSTRAINT_NAME,
    CONSTRAINT_TYPE
FROM
    INFORMATION_SCHEMA.TABLE_CONSTRAINTS
WHERE
    TABLE_SCHEMA = 'dima';
```

< 제약 조건의 확인 >

-- primary key와 NotNull 제약조건 확인

```
SELECT
    TABLE_NAME
    , COLUMN_NAME
    , IS_NULLABLE
    , COLUMN_KEY
FROM
    INFORMATION_SCHEMA.COLUMNS
WHERE
    TABLE_SCHEMA = 'dima'
    AND
    IS_NULLABLE = 'NO';
```

□ CREATE - Table

• 데이터베이스의 테이블의 정보 조회

SHOW TABLE status; -- 현재 사용중인 스키마 내의 모든 테이블 상태 조회
DESCRIBE employees; -- 지정된 테이블의 정보 조회

	ABC Name	ABC Engine	123 Version	ABC Row_format	123 Rows
1	countries	InnoDB	10	Dynamic	25
2	departments	InnoDB	10	Dynamic	27
3	employees	InnoDB	10	Dynamic	107
4	job_history	InnoDB	10	Dynamic	10
5	jobs	InnoDB	10	Dynamic	19
6	locations	InnoDB	10	Dynamic	23
7	parent	InnoDB	10	Dynamic	0
8	regions	InnoDB	10	Dynamic	4

	ABC Field	ABC Type	ABC Null	ABC Key	ABC Default	ABC Extra
1	employee_id	decimal(6,0)	NO	PRI	[NULL]	
2	first_name	varchar(20)	NO		[NULL]	
3	last_name	varchar(25)	NO		[NULL]	
4	email	varchar(25)	YES	UNI	[NULL]	
5	phone_number	varchar(20)	YES		[NULL]	
6	hire_date	date	NO		[NULL]	
7	job_id	varchar(10)	NO	MUL	[NULL]	
8	salary	decimal(8,2)	YES		[NULL]	
9	commission_pct	decimal(2,2)	YES		[NULL]	
10	manager_id	decimal(6,0)	YES	MUL	[NULL]	
11	department_id	decimal(4,0)	YES	MUL	[NULL]	

• 테이블의 생성

```
> CREATE TABLE table_name (
    col1 datatype 제약조건           -- 컬럼레벨 제약조건
    , col2 datatype 제약조건,
    , constraint 제약조건명 제약조건 -- 테이블레벨 제약조건
    ...
);
```

▣ CREATE - Table

• 제약조건 지정 방식

1) 컬럼 레벨 제약조건

- 선언한 컬럼의 타입 옆에 제약조건을 직접 기재하는 방식
- 제약조건명을 부여할 수 없으나 간단한 방법으로 제약조건을 줄 수 있다는 장점이 있다

2) 테이블레벨 제약조건

- 모든 컬럼을 정의하고 난 후 가장 마지막에 제약조건을 부여하는 방식
- 제약조건명을 부여할 수 있으므로 권장
- 제약조건이 부여되었을 경우 추후 ALTER 명령을 이용해 테이블의 구조를 변경해야 할 때 제약조건명을 이용할 수 있다.

• 제약조건명 지정

- 제약조건명을 부여하는 방식은 별다른 규칙이 없으나 주로 아래와 같이 명명하는 것이 좋다.

<테이블명_컬럼명_제약조건>

ex)

employee_eid_pk

department_dno_fk

▣ CREATE - Table

• 제약조건 확인

- 테이블에 제약조건의 종류나 이름을 확인하기 위해서는 information_schema 스키마의 table_constraints 또는 coumns 테이블에서 확인할 수 있다.

< 제약 조건의 확인 >

-- check, primary key, foreign key, unique 제약조건 확인

SELECT

TABLE_NAME,
CONSTRAINT_NAME,
CONSTRAINT_TYPE

FROM

INFORMATION_SCHEMA.TABLE_CONSTRAINTS

WHERE

TABLE_SCHEMA = '테이블명';

< 제약 조건의 확인 >

-- primary key와 NotNull 제약조건 확인

SELECT

TABLE_NAME
, COLUMN_NAME
, IS_NULLABLE
, COLUMN_KEY

FROM

INFORMATION_SCHEMA.COLUMNS

WHERE

TABLE_SCHEMA = '테이블명'

AND

IS_NULLABLE = 'NO';

▣ 테이블의 제약조건(Constraints)

• 제약조건

- 제약 조건은 테이블을 설계할 때 데이터베이스 내의 각각의 데이터의 일관성과 무결성을 지키기 위해 컬럼 값을 제한하는 조건을 말한다.
- 제약 조건을 적절히 사용함으로써 데이터의 유효성, 일관성 및 무결성을 유지할 수 있다.

※ 데이터 무결성(Data Integrity)

데이터가 전송, 저장되고 처리되는 모든 과정에서 변경되거나 손상되지 않고 완전성, 정확성, 일관성을 유지함을 보장하는 특성.

데이터 무결성은 데이터를 저장, 처리, 전송, 검색하는 모든 시스템에서 중요한 특성으로 특히 데이터베이스 시스템(database system)에서는 필수적인 기능 요소다.

데이터베이스 시스템에서는 무결성 제약 조건(integrity constraint)을 부여하여 데이터에 대한 결손과 부정합이 없음을 보증한다.

출처: 정보통신 용어사전

▣ 테이블의 제약조건(Constraints)

• 제약조건의 종류

종류	종류
PRIMARY KEY	테이블에 저장된 각 행을 유일하게 식별할 수 있도록 하는 제약조건 Primary Key는 Unique, Not Null 속성을 모두 가지며 테이블에는 Primary Key를 갖는 컬럼이 0~1개 있어야 한다.
FOREIGN KEY	다른 테이블과 관계(Relation)를 맺기 위한 제약조건. 관계를 맺은 두 테이블을 부모와 자식의 관계라고 한다. <u>Primary Key를 갖는 테이블이 부모 테이블이고 부모테이블의 Primary Key는 자식 테이블의 Foreign Key로 전이된다.</u> 외래키를 갖는 컬럼의 값을 이용해 부모 테이블의 행을 식별할 수 있으며 관계를 맺은 두 테이블들 간에는 참조 무결성이 적용된다. ※ 참조 무결성 (referential integrity): 관계형 데이터베이스에서 관련 있는 두 개의 테이블 간의 일관성(데이터 무결성)을 말한다. 부모 테이블의 특정 행을 삭제 시도하면 그 행의 Primary Key를 Foreign로 참조하는 자식 테이블에 문제가 생기므로 삭제가 되지 않도록 하거나 한꺼번에 삭제하도록 하여 데이터의 무결성과 일관성을 유지할 수 있다. on delete cascade → 부모테이블의 특정 행을 삭제하면 그 행을 참조하고 있던 자식테이블의 행도 함께 삭제 on update cascade → 부모테이블의 특정 행의 PK를 수정하면 그 행을 참조하고 있던 자식테이블의 FK도 함께 수정

▣ 테이블의 제약조건(Constraints)

• 제약조건의 종류

종류	종류
NOT NULL	컬럼에 NULL값이 입력되지 않도록 하는 제약조건 회원 테이블에서 이름 컬럼에 이름 값이 지정되지 않으면 제약조건에 위배된다. MySQL에서는 NOT NULL 제약조건은 <u>제약조건명을 지정할 수 없다</u>. 즉, 컬럼 레벨 제약조건만 부여가능
UNIQUE	컬럼의 전체 행 값에 중복되는 값이 없도록 하는 제약조건 회원테이블에서 이메일과 같은 값은 중복될 수 없다. Unique 제약조건을 갖는 컬럼은 Null은 중복해서 가질 수 있고, 해당 컬럼의 값을 Foreign Key로 전이할 수도 있다.
CHECK	조건으로 설정된 데이터의 값만 입력 허용하도록 하는 제약조건 회원 테이블의 성별을 입력해야 하는 컬럼이 있을 경우 '남', '여' 와 같이 정해진 값 만을 입력하도록 제약하며 '남성'이나 'Male' 등이 입력되지 않도록 하여 일관성을 유지한다.
DEFAULT (제약조건 아님)	데이터가 명시적으로 제공되지 않았을 때 <u>컬럼의 기본값</u>을 설정하기 위해 사용된다. DEFAULT는 제약 조건은 아니지만 제약 조건을 함께 사용하여 데이터베이스의 데이터 무결성과 일관성을 유지하면서 유연성을 높일 수 있다.

- 제약조건은 Create 명령으로 테이블을 생성할 때 지정할 수도 있고, 이미 생성된 테이블은 Alter 명령으로 제약조건을 수정 또는 추가, 삭제할 수 있다.

MySQL 문자 자료형

자료형		크기	설명
<u>char(n)</u>		1~255	고정길이 문자열. char 로 기재하며 char(1)과 동일
<u>varchar(n)</u>		1~65535	가변길이 문자열. 최대 65535크기의 문자열을 저장할 수 있다.
Text	tinytext	1~255	255크기의 text 데이터
	text	1~65535	최대 65535 크기의 text 데이터
	mediumtext	1~16777215	최대 16777215 크기의 text 데이터
	<u>longtext</u>	1~4294967295	최대 4GB 크기의 text 데이터
Blob	tinyblob	1~255	255크기의 blob 데이터
	blob	1~65535	최대 65535 크기의 blob 데이터
	mediumblob	1~16777215	최대 16777215 크기의 blob 데이터
	<u>longblob</u>	1~4294967295	최대 4GB 크기의 blob 데이터
binary(n)		1~255	고정 길이 이진 데이터
varbinary(n)		1~255	가변 길이 이진 데이터
enum(값...)		1 또는 2	최대 65535개의 열거형 데이터 값들
set(값...)		1, 2, 3, 4, 8	최대 64개 다른 데이터 값들

MySQL 숫자 자료형

자료형	크기(바이트)	값의 범위	설명
bit(n)	N/8	bit(m) : (m: 1~64)	1~64bit를 표현 b'0000'형식으로 표현. 비트값을 저장
tinyint	1	-128~127	정수. bool 자료형이 있으나 ENUM(Y,N) 또는 TINYINT(1) 로 대체하여 사용하는 것을 권장
<u>smallint</u>	2	-32,768 ~ +32,767	정수
mediumint	3	-8,388,608 ~ +8,388,607	정수
<u>int</u> , integer	4	$-2^{31} - 1 \sim 2^{31}$	정수
bigint	8	$-2^{63} - 1 \sim 2^{63}$	정수
float	4	-3.4E-38 ~ 1.17E+38	부동 소수점수. 소수점 이하 7자리까지 표현. MySQL 8.0.17 이후 버전부터 사용되지 않음.
double, real	8	-1.22E-308 ~ 1.79E+308	부동 소수점수. 소수점 이하 15자리까지 표현 가능 MySQL 8.0.17 이후 버전부터 사용되지 않음.
<u>decimal(m, d)</u> , numeric(m, d)	5~17	$-10^{38} + 1 \sim 10^{38} - 1$	고정 소수점수 전체 자릿수(m), 소수이자 자릿수(d)를 가진 자료형 d 값이 0이면 소수 자리가 없음

※ 숫자 자료형은 부호 여부에 따라 signed, unsigned로 나뉜다.

MySQL 날짜, 시간 자료형

자료형	크기(바이트)	설명
date	3	1001-01-01 ~ 9999-12-31까지 저장. 기본형식 'YYYY-MM-DD'
time	3	시간을 저장할 수 있는 타입. 기본 형식은 'HH:MM:SS'
datetime	8	날짜와 시간을 저장할 수 있는 타입. 기본 형식은 'YYYY-MM-DD HH:MM:SS' 1001-01-01 00:00:00 ~ 9999-12-31 23:59:59 까지 저장.
timestamp	4	날짜와 시간을 나타내는 타임스탬프를 저장할 수 있는 타입 1001-01-01 00:00:00 ~ 9999-12-31 23:59:59 까지 저장. time_zone 시스템 변수와 관련이 있으며 UTC 시간대 변환하여 저장
year	1	연도를 저장할 수 있는 타입. year(2), year(4) 방식은 사용을 권장하지 않음 참고: MySQL 5.7 Reference > 2-Digit YEAR(2) Limitations and Migrating to 4-Digit YEAR 1901~2155까지 저장 'YYYY' 형식으로 사용

[실습] Date타입의 사용

```
CREATE TABLE timetest (
  col1 date
, col2 datetime
, col3 timestamp
, col4 time
, col5 year
);
```

```
INSERT timetest
VALUES (
  CURRENT_TIMESTAMP
, CURRENT_TIMESTAMP
, CURRENT_TIMESTAMP
, CURRENT_TIMESTAMP
, CURRENT_TIMESTAMP
);
```

```
SELECT * FROM timetest;
```

col1	col2	col3	col4	col5
2024-05-28	2024-05-28 17:04:46	2024-05-28 17:04:46	17:04:46	2024

타입에 따라 같은 값이라도 입력 값이 다르다

□ 자료형 변환

• 명시적 형 변환

- cast(), convert() 함수를 이용하여 데이터의 타입을 바꿀 수 있다.
- 이 함수로 변경하는 방법을 명시적 형변환이라고 한다.
- binary, char, date, datetime, decimal, json, signed integer, time, unsigned integer 타입을 형 변환 할 수 있다.

CAST (expression AS 데이터형식 [(길이)]);

CONVERT (expression , 데이터형식 [(길이)]);

```
SELECT avg(amount) AS '평균 구매 수량' FROM payment p;           -- 4.201356
```

```
SELECT CONVERT(avg(amount), signed integer)
      AS '평균 구매 수량' FROM payment p;                       -- 4
```

```
SELECT rental_id,
       concat(cast(avg(amount) * 10 AS char(4)), '개') AS '평균구매수량'
FROM payment p
GROUP BY rental_id ;
```

```
SELECT cast('2020-12-12' AS date);
```

	rental_id	평균구매수량
1	1	29.9개
2	2	29.9개
3	3	39.9개
4	4	49.9개
5	5	69.9개
6	6	9.90개
7	7	19.9개
8	8	49.9개
9	9	49.9개

□ 자료형 변환

• 묵시적 형 변환

- 앞에서 사용했던 cast(), convert() 함수를 이용하지 않고 연산 시 형이 변환되는 것.
- 산술 연산자(+, -, *, /, %)는 피연산자의 데이터 타입이 숫자로 변환이 가능한 경우 변경되어 연산
- 비교 연산자(> < >= <= !=)는 문자열 자체로 비교

```
SELECT '100' + '150' ;           -- 250 (숫자로 변경되어 연산)
SELECT '2020' + 1;               -- 2021 ('2020'이 숫자로 변경되어 연산)
SELECT '100' < '50';              -- 1 (=true, 문자열 '1'과 '5'를 비교)

SELECT 'A' = 'a';                 -- 1 (true). 대소문자를 구분하여 비교하지 않음
SELECT 'b' > 'a';                 -- 1 (true)

SELECT BINARY 'A' = BINARY 'a'; -- 0 (false) 대소문자를 구분하여 비교
```

▣ 테이블의 제약조건(Constraints)

- Foreign Key 정의

< 테이블 생성 시 외래키 추가: 테이블 레벨 >

```
CREATE TABLE table_name (  
    id INT PRIMARY KEY,  
    col2 CHAR (20),  
    col3 INT,  
    ...  
    CONSTRAINT col3_fk FOREIGN KEY(col3)  
    REFERENCING parent_table_name(UNIQUE(key_col) ON DELETE CASCADE,  
    ... )
```

< 테이블 생성 시 외래키 추가: 컬럼 레벨 >

```
CREATE TABLE table_name (  
    id INT PRIMARY KEY,  
    col2 CHAR(20),  
    col3 INT FOREIGN KEY REFERENCES parent_table_name(column_name),  
    ... )
```

▣ 테이블의 제약조건(Constraints)

- Foreign Key 정의

< 테이블 수정 시 외래키 추가 >

```
ALTER TABLE <table identifier>  
  ADD [ CONSTRAINT <constraint identifier> ]  
    FOREIGN KEY ( <column expression> {, <column expression>}... )  
    REFERENCES <table identifier> [ ( <column expression> {, <column expression>}... ) ]  
    [ ON UPDATE <referential action> ]  
    [ ON DELETE <referential action> ]
```

- Foreign Key의 확인

< 테이블에 지정된 constraint 내용 확인 >

```
SELECT * FROM information_schema.table_constraints [ WHERE table_name = '테이블명' ];
```

▣ 테이블의 제약조건(Constraints)

- 제약조건 확인

< 제약 조건의 확인 >

-- check, primary key, foreign key, unique 제약조건 확인

SELECT

TABLE_NAME,
CONSTRAINT_NAME,
CONSTRAINT_TYPE

FROM

INFORMATION_SCHEMA.TABLE_CONSTRAINTS

WHERE

TABLE_NAME = '테이블명';

< 제약 조건의 확인 >

-- primary key와 not null 제약조건 확인

SELECT

TABLE_NAME
, COLUMN_NAME
, IS_NULLABLE
, COLUMN_KEY

FROM

INFORMATION_SCHEMA.COLUMNS

WHERE

TABLE_SCHEMA = '테이블명'

AND

IS_NULLABLE = 'NO';

▣ 테이블의 제약조건(Constraints)

- Foreign Key 삭제

< 외래키 삭제 >

ALTER TABLE <table identifier>

DROP [CONSTRAINT <constraint identifier>]

FOREIGN KEY (<column expression> {, <column expression>}...)

REFERENCES <table identifier> [(<column expression> {, <column expression>}...)]

[ON UPDATE <referential action>]

[ON DELETE <referential action>]

▣ Index 생성 및 삭제

- Index 생성

```
CREATE [UNIQUE | FULLTEXT | SPATIAL ] INDEX index_name  
[index_type]  
ON tbl_name (key_part, ...)  
[index_option]  
[algorithm_option | lock_option] ...
```

- Index 제거

```
DROP INDEX index_name ON tbl_name  
[algorithm_option | lock_option] ...
```

▣ CREATE - Index

-- 1) 테이블 생성

CREATE TABLE users

(

uid int AUTO_INCREMENT

, name varchar(20) NOT NULL

, phone varchar(25) NOT NULL

, age decimal(3) DEFAULT 20

, gender char(4) DEFAULT '남성'

, CONSTRAINT parent_id_pk PRIMARY KEY (uid)

, CONSTRAINT parent_phone_uq UNIQUE (phone)

, CONSTRAINT parent_age_ck CHECK (age >= 20)

, CONSTRAINT parent_gender_ck CHECK (gender IN ('남성', '여성'))

);

-- 2) index 확인하기 (pk와 uq 제약조건은 자동으로 index가 생성된다.)

SHOW INDEX FROM users;

	ABC Table ▼	123 Non_unique ▼	ABC Key_name ▼	123 Seq_in_index ▼
1	users	0	PRIMARY	1
2	users	0	parent_phone_uq	1
3	users	1	user_index_ix	1

□ VIEW

- 뷰의 생성 및 사용

<View의 생성>

```
CREATE VIEW view_name  
AS  
    SELECT 구문;
```

<View의 사용>

```
SELECT 컬럼명 FROM view_name;
```

▣ ALTER

- Alter 명령

- 테이블의 구조를 변경하거나 제약조건을 변경할 때 사용하는 명령이다.

- 테이블의 구조 변경

-- 테이블 이름 변경

ALTER TABLE 테이블이름 RENAME TO 새로운테이블명;

-- 컬럼 추가 (add)

ALTER TABLE 테이블이름 ADD 컬럼이름 컬럼타입

-- 컬럼을 추가할 때 기존의 어느 컬럼 뒤에 추가하는지 위치 지정

ALTER TABLE 테이블이름 ADD 컬럼이름 컬럼타입 AFTER 기존컬럼명

-- 컬럼 제거 (drop)

ALTER TABLE 테이블이름 DROP 컬럼이름

-- 컬럼 수정 (change)

ALTER TABLE 테이블이름 CHANGE 컬럼명 새컬럼명 새컬럼타입

-- 컬럼 타입만 수정 (modify)

ALTER TABLE 테이블이름 MODIFY 컬럼명 새컬럼타입

▣ ALTER

- Alter를 이용해 테이블의 구조를 변경할 때 오류가 발생하는 경우 주의할 점

- 변경하고자 하는 열에 이미 존재하는 데이터가 수정하려는 데이터 타입의 제약 조건을 충족하지 않는 경우.
- 변경하고자 하는 열이 PK 제약조건이 있어 외래 키로 다른 테이블과 연결되어 있는 경우 오류가 발생하며 이 경우에는 추가적인 제약 조건을 처리해야 한다.

- 제약조건 설정 / 변경

```
-- 제약조건 확인하기
```

```
DESC 데이터베이스 명.테이블 명;
```

```
-- 테이블 이름 변경
```

```
ALTER TABLE [테이블 명] RENAME TO [새로운 테이블 명] ;
```

```
-- 기본키 추가:
```

```
ALTER TABLE [테이블 명] ADD CONSTRAINT [제약조건 이름] PRIMARY KEY(컬럼 명);
```

```
-- 특정 컬럼의 타입 변경
```

```
ALTER TABLE [테이블 명] MODIFY [컬럼명] [새로운 타입] ;
```

■ ALTER

• 테이블의 제약조건 설정 / 변경

-- NOT NULL 제약 조건 추가 : NOT NULL 제약조건은 ADD는 안되고 MODIFY로 제약조건을 추가/변경해야 한다.

ALTER TABLE [테이블 명] **MODIFY** [컬럼 명] [데이터 타입] **CONSTRAINT** [제약조건 이름] **NOT NULL**;

-- 외래키 추가

ALTER TABLE [테이블 명] **ADD CONSTRAINT** [제약조건 이름] **FOREIGN KEY**(컬럼 명)
REFERENCES [부모테이블 명](PK 컬럼 명) [**ON DELETE CASCADE** / **ON UPDATE CASCADE**];

-- CHECK 추가 :

ALTER TABLE [테이블 명] **ADD** 컬럼명 [데이터 타입] **CHECK** (컬럼명 **IN** ('값1 ', ' 값2 ', ' 값3 '));

-- CHECK 추가: 특정 컬럼 뒤에 추가

ALTER TABLE [테이블 명] **ADD** 컬럼명 [데이터 타입] **CHECK** (컬럼명 **IN** (('값1 ', ' 값2 ', ' 값3 ')) **AFTER** 컬럼명;

-- UNIQUE 추가 : 기존에 unique 제약조건이 걸려 있으면 제약조건을 modify하는 것은 안됨

ALTER TABLE [테이블 명] **ADD CONSTRAINT** [제약조건 이름] **UNIQUE**(컬럼명);

-- 제약조건 삭제

ALTER TABLE [테이블 명] **DROP CONSTRAINT** [제약조건 이름];

ALTER TABLE [테이블 명] **DROP FOREIGN KEY** [제약조건 이름];

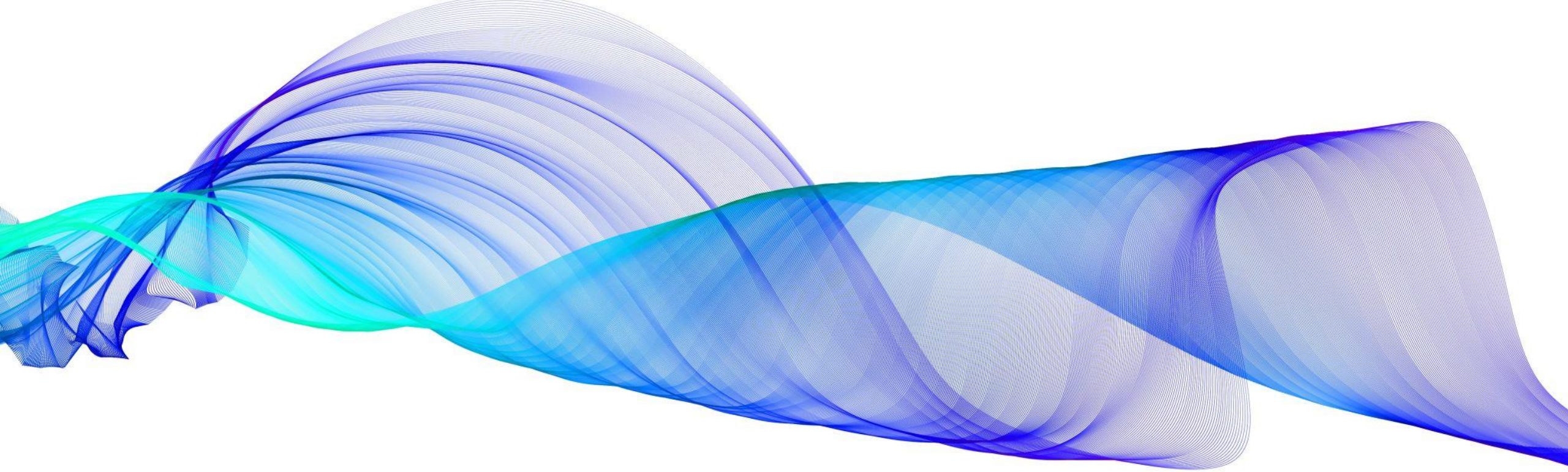
□ DROP

- DROP 명령

- 사용자, 테이블, 인덱스 등 MySQL이 관리하는 개체를 삭제할 때 사용하는 명령이다.
- 테이블을 DROP 할 때에는 DROP 할 테이블의 자식테이블에 데이터가 존재하면 실행할 수 없다.
즉, 자식테이블 먼저 DROP한 후에 부모 테이블 DROP

```
# DROP 명령의 사용  
DROP TABLE 테이블명;
```

```
DROP TABLE person;  
DROP TABLE grade;
```



Transaction

□ Transaction

- **트랜잭션(Transaction)이란?**

- 데이터베이스 시스템에서 사용되는 업무처리의 단위이다.
- 영어의 Transaction은 거래를 뜻한다. 예를 들어 돈을 주었는데 물건을 받지 못한다면, 그 거래는 이루어지지 못하고 원상태로 복구되어야 한다. 이와 같이 쪼갤 수 없는 하나의 처리 행위를 원자적 행위라고 한다. 여기서 쪼갤 수 없다는 말의 의미는 실제로 쪼갤 수 없다는 의미라기 보다는 만일 쪼개질 경우 시스템에 심각한 오류를 초래할 수 있다는 것이다.
- 이러한 개념의 기능을 ATM 또는 데이터베이스 등의 시스템에서 제공하는 것이 바로 트랜잭션이다.
- 트랜잭션(Transaction) 관련 명령어는 데이터베이스 내에서 일련의 작업을 하나의 단위로 묶어 실행하고, 필요한 경우 전체 작업을 취소할 수 있도록 한다.
- 트랜잭션은 데이터의 일관성과 무결성을 유지하는 데 중요한 역할을 한다.

□ Transaction

- **START TRANSACTION / BEGIN**

- 트랜잭션의 시작. 두 명령어는 동일

```
START TRANSACTION;  
또는  
BEGIN;
```

- **COMMIT**

- 트랜잭션 내에서 수행된 모든 변경 사항을 영구적으로 저장. 트랜잭션이 성공적으로 완료되었음을 나타낸다.

```
COMMIT;
```

- **ROLLBACK**

- 트랜잭션 내에서 수행된 모든 변경 사항을 취소하고, 트랜잭션 시작 이전 상태로 되돌린다. 트랜잭션이 실패하거나 취소되었음을 나타낸다.

```
ROLLBACK;
```

□ Transaction

- **SAVEPOINT**

- 트랜잭션 내에서 특정 시점(저장 지점)을 설정한다. 이렇게 설정된 지점까지 Rollback할 수 있다.

```
SAVEPOINT savepoint_name;
```

- **ROLLBACK TO SAVEPOINT**

- SAVEPOINT로 지정한 위치까지 Rollback 한다. Transaction의 일부만 취소한다.

```
ROLLBACK TO SAVEPOINT savepoint_name;
```

- **RELEASE SAVEPOINT**

- 더 이상 필요 없는 저장 지점을 삭제한다. 저장 지점이 삭제되면 해당 지점으로 Rollback 할 수 없다.

```
RELEASE SAVEPOINT savepoint_name;
```

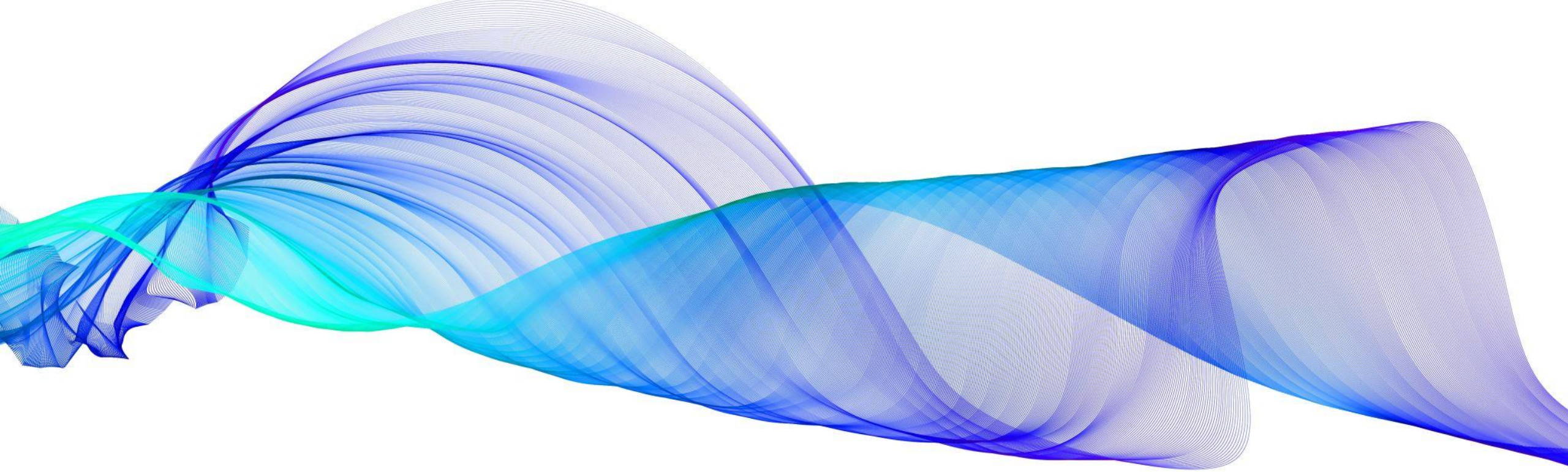
□ Transaction

- SET AUTOCOMMIT

- 자동 커밋 모드를 설정하거나 해제. MySQL은 기본적으로 auto commit mode가 활성화되어 있다.

```
SET AUTOCOMMIT = {0 | 1};
```

- SET AUTOCOMMIT = 0; : 자동 커밋 비활성화(수동으로 COMMIT 명령을 실행해야 변경 사항이 저장됨)
- SET AUTOCOMMIT = 1; : 자동 커밋 활성화 (각 SQL 문이 자동으로 commit됨)



DCL(Data Control Language)

▣ DCL (Data Control Language)

• DCL이란?

- 데이터베이스 사용자에게 특정 권한을 부여하거나 회수하는 데 사용되는 명령어.
 - DCL 명령어를 사용할 수 있는 권한은 주로 데이터베이스 관리자(DBA)나 고급 사용자에게 주어진다.
 - 권한 부여와 회수는 데이터베이스 보안과 직접적으로 관련되므로 신중하게 관리해야 한다.
 - DCL 명령어들은 MySQL의 보안을 유지하고 데이터 접근을 제어하는 데 필수적인 도구이므로 적절히 활용하여 데이터베이스의 안전한 운영을 유지하는 것이 중요.
-
- 주요 DCL 명령어는 다음과 같다.
 - GRANT : 사용자에게 데이터베이스 객체에 대한 권한을 부여
 - REVOKE : 사용자에게 부여된 권한을 회수

GRANT privilege [, privilege] ...

ON object_type **TO user** [IDENTIFIED BY 'password'] [, user ...]

[WITH GRANT OPTION]; -- 'WITH GRANT OPTION'은 부여받은 권한을 다른 사용자에게 다시 부여할 수 있는 권한을 의미.

REVOKE privilege [, privilege] ...

ON object_type **FROM user** [, user ...];

▣ 권한의 부여와 회수

```
-- GRANT
-- 'some_user'에게 'database_name.table_name'에 대한 'SELECT'와 'INSERT' 권한을 부여.
GRANT SELECT, INSERT
ON database_name.table_name TO 'some_user'@'host_name' IDENTIFIED BY 'password';

-- WITH GRANT OPTION
-- 부여받은 권한을 다른 사용자에게 다시 부여 가능
GRANT SELECT ON database_name.table_name TO 'some_user'@'host_name' WITH GRANT OPTION;

-- REVOKE 예
-- 'some_user'에게 부여된 'database_name.table_name'에 대한 'SELECT'와 'INSERT' 권한을 회수
REVOKE SELECT, INSERT
ON database_name.table_name FROM 'some_user'@'host_name';
```

□ 권한의 종류

• 자주 사용되는 권한(Privileges)의 종류

권한	설명
ALL PRIVILEGES	모든 권한을 부여
SELECT	데이터를 조회할 수 있는 권한
INSERT	데이터를 삽입할 수 있는 권한
UPDATE	데이터를 수정할 수 있는 권한
DELETE	데이터를 삭제할 수 있는 권한
CREATE	데이터베이스와 테이블을 생성할 수 있는 권한
DROP	데이터베이스와 테이블을 삭제할 수 있는 권한
ALTER	테이블 구조를 변경할 수 있는 권한
INDEX	인덱스를 생성하고 삭제할 수 있는 권한
EXECUTE	저장 프로시저와 함수를 실행할 수 있는 권한

▣ User 생성 및 권한 부여

• User의 관리

- 사용자를 생성, 삭제, 권한 수정하는 명령어들은 데이터베이스의 보안과 관리에 필수적인 명령어들이다.

-- 1. 사용자 생성 : 새로운 사용자를 생성하고 비밀번호를 설정.

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

-- 2. 사용자 삭제 : 기존 사용자를 삭제.

```
DROP USER 'username'@'host';
```

-- 3. 사용자 권한 부여 : 특정 사용자에게 특정 데이터베이스나 테이블에 대한 권한을 부여.

```
GRANT privilege [, privilege] ...
```

```
ON database_name.table_name
```

```
TO 'username'@'host';
```

-- 4. 사용자 권한 회수 : 특정 사용자에게 부여된 권한을 회수하는 명령어입니다.

```
REVOKE privilege [, privilege] ...
```

```
ON database_name.table_name
```

```
FROM 'username'@'host';
```

-- 모든 권한 회수

```
REVOKE ALL PRIVILEGES ON mydatabase.* FROM 'newuser'@'localhost';
```

▣ User 생성 및 권한 부여

```
-- 5. 권한 적용
-- 'GRANT'와 'REVOKE' 명령어는 권한 테이블에 변경 사항을 적용하기 위해 'FLUSH PRIVILEGES'. (MySQL 8.0 이상에서는 불필요)
FLUSH PRIVILEGES;

-- 6. 사용자 권한 확인
-- 특정 사용자의 권한을 확인
SHOW GRANTS FOR 'username'@'host';
```

▣ User 생성 및 권한 부여

-- 모든 데이터베이스의 모든 테이블에 모든 권한 부여

GRANT ALL PRIVILEGES ON *.* TO '사용자'@'localhost';

-- 특정 데이터베이스의 모든 테이블에 모든 권한 부여

GRANT ALL PRIVILEGES ON DB이름.* TO '사용자'@'localhost';

-- 특정 데이터베이스의 특정 테이블에 모든 권한 부여

GRANT ALL PRIVILEGES ON DB이름.테이블명 TO '사용자'@'localhost';

-- 특정 데이터베이스의 특정 테이블에 select 권한 부여

GRANT SELECT ON DB이름.테이블명 TO '사용자'@'localhost';

-- 특정 데이터베이스의 특정 테이블에 select, insert 권한 부여

GRANT SELECT, INSERT ON DB이름.테이블명 TO '사용자'@'localhost';

-- 특정 데이터베이스의 특정 테이블의 컬럼1과 컬럼2의 update 권한 부여

GRANT UPDATE(컬럼1, 컬럼2) ON DB이름.테이블명 TO '사용자'@'localhost';

□ User 생성 및 권한 부여

- 모든 MySQL 사용자 표시
 - MySQL은 사용자에게 대한 정보를 테이블에 저장한다.

```
SELECT user, host FROM mysql.user;
```

	user	host
1	mysql.infoschema	localhost
2	mysql.session	localhost
3	mysql.sys	localhost
4	root	localhost
5	scott	localhost

The End!